POLITECHNIKA GDAŃSKA
WYDZIAŁ ELEKTRONIKI, TELEKOMUNIKACJI I INFORMATYKI

Imię i nazwisko studenta: Jakub Ławicki
Nr albumu: 189788
Poziom kształcenia: Studia pierwszego stopnia
Forma studiów: stacjonarne
Kierunek studiów: Automatyka, cybernetyka i robotyka
Profil: Systemy decyzyjne i robotyka

PROJEKT DYPLOMOWY INŻYNIERSKI

Tytuł projektu w języku polskim: Implementacja i badania samoorganizujących się sztucznych sieci neuronowych typu WTM w zadaniach klasyfikacji danych

Tytuł projektu w języku angielskim: Implementation and research of self-organizing artificial neural networks of the WTM type in data classification tasks

Opiekun pracy: dr hab. inż. Tomasz Talaśka

STRESZCZENIE

Sieci samoorganizujące (SOM), nazywane również sieciami Kohonena, są przykładem sztucznych sieci neuronowych uczonych bez nauczyciela. Jakkolwiek nie cieszą się one dużą popularnością, to wykorzystuje się je niekiedy jako narzędzie redukcji wymiarów danych wejściowych. W prezentowanej pracy zbadano możliwości zastosowania sieci SOM w zadaniach związanych z klasyfikacją danych. Problem ten odgrywa kluczową rolę w automatyce i robotyce, ponieważ umożliwia podejmowanie trafnych decyzji, w oparciu o bieżące informacje.
W pracy zawarto zarówno teoretyczny opis sztucznych sieci neuronowych, jak i sieci samoorganizujących się. Ponadto przedstawiono rozwiązanie implementujące sieci SOM z algorytmem WTM, zaimplementowane w języku Python oraz omówiono przeprowadzone badania na trzech ogólnodostępnych zbiorach danych: MNIST, IRIS i Cardiotocography, wraz z opisem wyników.
W przeprowadzonych badaniach szczególny nacisk położono na badanie wpływu zastosowanej funkcji sąsiedztwa na proces uczenia sieci, w oparciu o analizę błędu kwantyzacji i błędu topograficznego. Uzyskane rezultaty wskazały na skuteczność sieci SOM w procesie klasyfikacji danych wejściowych, w przypadku zbiorów o niewielkiej liczbie wymiarów. W przypadku bardziej złożonych zbiorów uzyskane wyniki były mniej obiecujące, choć nadal akceptowalne, wskazując na możliwość przeprowadzenia badań, w których sieci samoorganizujące się zostaną uzupełnione o inne techniki przetwarzania danych.

Słowa kluczowe: sieci samoorganizujące się, sieci Kohonena, algorytm WTM, klasyfikacja danych, klasteryzacja

Dziedzina nauki i techniki zgodna z OECD Nauki inżynieryjne i techniczne, Elektrotechnika, elektronika i inżynieria informatyczna, Robotyka i Automatyka

ABSTRACT

Self-organizing maps, also known as Kohonen networks, are an example of artificial neural networks trained using unsupervised learning. They are not widely popular, but they are occasionally used for reducing the dimensionality of input data.This study explores the potential application of Self-Organizing Maps (SOM) networks in data classification tasks. This problem has a key role in the fields of automation and robotics, as it facilitates precise decision-making based on current information.

The paper includes both a theoretical description of artificial neural networks and self-organizing maps. In addition, it presents a solution implementing SOM networks with the WTM algorithm, implemented in Python, and discusses the research conducted on three publicly available datasets: MNIST, IRIS and Cardiotocography, along with a description of the results.

In the conducted research, special emphasis was placed on studying the impact of the applied neighborhood function on the learning process of the network, based on the analysis of quantization error and topographic error. The obtained results indicated the effectiveness of the SOM network in the process of input data classification, in the case of sets with a small number of dimensions. In the case of more complex sets, the results obtained were less promising, although still acceptable, indicating the possibility of conducting research in which self-organizing networks will be supplemented with other data processing techniques.

Keywords: Self-organizing maps, unsupervised learning, clustering, classification, WTM algorithm

OECD consistent field of science and technology classification: Engineering and technology, Electrical engineering, Electronic engineering, Information engineering, Robotics and automatic control

SPIS TREŚCI

Spis treści **5**

1 Wstęp i cel pracy **8**
1.1 Cel pracy 8
1.2 Zawartość rozdziałów 8

2 Teoretyczny opis sztucznych sieci neuronowych **10**
2.1 Opis sztucznego neuronu 10
2.2 Klasyczny model sztucznych sieci neuronowych 13
2.3 Uczenie sieci neuronowych 15
2.4 Podsumowanie rozdziału 17

3 Samoorganizujące się sztuczne sieci neuronowe **18**
3.1 Opis sieci samoorganizujących się 19
3.2 Algorytm Zwycięzca bierze większość (WTM) 20
3.2.1 Opis algorytmu 20
3.2.2 Topologia samoorganizującej się sieci neuronowej 21
3.2.3 Funkcje sąsiedztwa 22
3.2.4 Adaptacja parametrów w czasie 24
3.3 Algorytm WTA 24
3.3.1 Mechanizm Sumienia 25
3.4 Algorytm gazu Neuronowego 26
3.5 Miary organizacji sieci 27
3.6 Przykłady możliwości zastosowania sieci SOM 28
3.7 Podsumowanie rozdziału 29

4 Proponowana implementacja sieci SOM **30**
4.1 Dostępne implementacje programowe sieci SOM 30
4.2 Wykorzystane narzędzia programistyczne 31
4.3 Opis implementacji programowej 32
4.4 Opis parametrów sieci SOM wraz z zasadami doboru 35
4.5 Podsumowanie rozdziału 37

5 Przeprowadzone badania **38**
5.1 Metodologia badań 38
5.2 Badania na zbiorze IRIS 39
5.2.1 Wnioski z badań na zbiorze IRIS 40
5.3 Badania na zbiorze Cardiotocography 41
5.3.1 Wnioski z badań na zbiorze Cardiotocography 42
5.4 Badania na zbiorze MNIST 43
5.4.1 Wnioski z badań na zbiorze MNIST 44

6 Podsumowanie **47**
6.1 Wnioski z przeprowadzonych badań 47
6.2 Potencjalne kierunki dalszych prac 48

Spis rysunków **49**

Spis tabel **50**

Dodatek A Instrukcja dla użytkownika **53**

LISTA SYMBOLI

v - całkowita wartość pobudzenia neuronu
W - wektor wag sieci neuronowej
X - wektor wejściowy
b - bias (składowa stała)
x_i - *i*-te wejście
w_i - waga przypisana *i*-temu wejściu
y - wartość wyjściowa neuronu
$f(\cdot)$ - funkcja aktywacji
β - parametr odpowiadający za kształt charakterystyki sigmoidalnej funkcji aktywacji
Δ - opóźnienie wyjścia podane na wejście w następnym kroku sieci rekurencyjnej.
n - liczba wymiarów danych
X - wektor wejściowy
$D(X, W_j)$ - odległość między wektorem wejściowym a wektorem wag j-tego neuronu w sieci SOM
W_j - *n*-wymiarowy wektor wag j-tego neuronu
$W_j(l)$ - wektor wag neuronu w iteracji l
$\eta(k)$ - współczynnik uczenia w epoce k
R - promień sąsiedztwa
$d(i, j)$ - odległość topologiczna między neuronami
$G(R, d(i, j))$ - wartość funkcji sąsiedztwa
$p_i(l)$ - potencjał neuronu stosowany w algorytmie WTA
p_{min} - minimalny potencjał uprawniający do wzięcia udziału w rywalizacji podczas algorytmu WTA
h - indeks przypisywany podczas algorytmu gazu neuronowego
$G(f)$ - funkcja sąsiedztwa w algorytmie gazu neuronowego
λ - parametr z algorytmu gazu neuronowego odpowiadający promieniowi sąsiedztwa z WTM
X_i - pojedynczy wzorzec wejściowy
$Q(X_i)$ - błąd kwantyzacji dla pojedynczego neuronu
Q_{error} - średni błąd kwantyzacji, nazywany w pracy błędem kwantyzacji
N - liczba wektorów wejściowych w zbiorze danych
Q_{max} - maksymalny błąd kwantyzacji dla danego zbioru danych
T_{error} - błąd topograficzny
h_k - funkcje zmiany parametrów sieci SOM
A - początkowa wartość promienia sąsiedztwa/współczynnika uczenia, w funkcji h_k
T - całkowita liczba epok treningowych
C - stała czasowa w funkcji zmiany parametrów

LISTA SKRÓTÓW

SOM - samoorganizująca się sieć neuronowa (ang. *Self-Organizing Maps*)
ANN - sztuczne Sieci Neuronowe (ang. *Artificial Neural Networks*)
FNN - sieci Neuronowe jednokierunkowe (ang. *Feedforward Neural Networks*)
RNN - rekurencyjne Sieci Neuronowe (ang. *Recurrent Neural Networks*)
WTA - algorytm Zwycięzca Bierze Wszystko (ang. *Winner Takes All*)
WTM - algorytm Zwycięzca Bierze Większość (ang. *Winner Takes Most*)
SOFM - samoorganizująca się sieć neuronowa (ang. *Self-Organizing Feature Map*)
BMU - neuron zwycięzca (ang. *Best Matching Unit*)
LVQ - kwantyzacja wektorowa (ang. *Learning Vector Quantization*)
E-DSOM - rozszerzone głębokie mapy sOM (ang. *Extended Deep SOM*)
DeepSOM - głębokie mapy smoorganizujące (ang. *Deep Self-Organizing Maps*)
MIT - licencja otwartego oprogramowania.
GPU - procesor graficzny (ang. *Graphics Processing Unit*)
CPU - procesor centralny (ang. *Central Processing Unit*)
MNIST - zbiór danych ręcznie pisanych cyfr (ang. *Modified National Institute of Standards and Technology*)
hex_{ncp} - topologia heksagonalna z niecałkowitymi wartościami promienia
$rect_{ncp}$ - topologia prostokątna z niecałkowitymi wartościami promienia
hex_{cp} - topologia heksagonalna z całkowitymi wartościami promienia
$rect_{cp}$ - topologia prostokątna z całkowitymi wartościami promienia

1. WSTĘP I CEL PRACY

Sztuczne sieci neuronowe są narzędziem niezwykle popularnym, wykorzystywanym do rozwiązywania różnorodnych problemów, zarówno w środowisku akademickim, jak i w przemyśle. Możliwości ich zastosowania są praktycznie nieograniczone i zależą przede wszystkim od wyobraźni projektanta oraz dostępnych zasobów obliczeniowych.
Jednym z podstawowych problemów, rozwiązywanych z pomocą sztucznych sieci neuronowych, jest klasyfikacja danych wejściowych. Jest to zadanie praktyczne i powszechnie spotykane, polegające na przykład na przypisywaniu danych liczbowych, takich jak wyniki badań medycznych czy sprawozdań finansowych do zdefiniowanych kategorii. Innym przykładem zastosowania jest klasyfikacja obrazów, spotykana często w diagnostyce medycznej lub w rozpoznawaniu obiektów w systemach wizyjnych w automatyce przemysłowej.

Przy rozwiązywaniu zadania klasyfikacji danych użyteczne mogą okazać się samoorganizujące się sieci neuronowe, nazywane również sieciami Kohonena. Są one przykładem sieci uczonych bez nadzoru, co oznacza, że przetwarzają dane bez konieczności posiadania informacji o wartości oczekiwanej (tj. przynależności danych uczących do konkretnych klas). Pomimo tej istotnej zalety nie są one jednak tak popularne jak inne sieci neuronowe, ze względu na ich potencjalne ograniczenia przy rozwiązywaniu bardzo złożonych problemów. W prezentowanej pracy zdecydowano się na sprawdzenie możliwości wykorzystania sieci SOM przy rozwiązywaniu problemu klasyfikacji.

1.1. Cel pracy

Celem pracy jest zbadanie wpływu różnych funkcji sąsiedztwa (Gaussa, trójkątnej, progowej i eksponencjalnej) na możliwości wykorzystania sieci SOM we wstępnym etapie procesu klasyfikacji danych, czyli klasteryzacji. Klasteryzację można przedstawić jako proces grupowania danych w zbiory, w których elementy są do siebie podobne względem obranego kryterium, a różnią się od elementów innych grup. W pracy zostanie zbadany dodatkowo wpływ wybranej topologii sieci.

Jakość przypisania do klas oceniona będzie za pomocą dwóch powszechnie stosowanych miar w sieciach Kohonena: błędu kwantyzacji i błędu topograficznego. Dodatkowo wykorzystane zostaną mapy zwycięzców, do wizualnej weryfikacji jakości grupowania danych. Będą one jednak traktowane jako narzędzia pomocnicze.

W celu realizacji postawionych założeń zostaną przeprowadzone badania implementujące sieci SOM w różnych kombinacjach funkcji sąsiedztwa, topologii sieci oraz sposobu określania wartości promienia sąsiedztwa. Sieci te zostaną poddane uczeniu na trzech zbiorach danych, różniących się między sobą wielkością, liczbą wymiarów oraz złożonością i strukturą klas.

1.2. Zawartość rozdziałów

Ogólną strukturę niniejszej pracy można przedstawić w następujący sposób:

- **Rozdział 2: Teoretyczny opis sztucznych sieci neuronowych.**
  Obejmuje podstawowe informacje na temat sztucznych sieci neuronowych, niezbędne do zrozumienia dalszej części pracy.
- **Rozdział 3: Opis samoorganizujących się sztucznych sieci neuronowych.**
  Omawia szczegółowo zasadę działania sieci samoorganizujących się, zawiera opis algo-

rytmu uczenia WTM oraz niezbędne pojęcia takie jak funkcje sąsiedztwa oraz topologię sieci. Przedstawia zastosowania sieci Kohonena oraz inne algorytmy jej nauki.

- **Rozdział 4: Proponowana implementacja sieci SOM.**
  Opisuje zaimplementowaną sieć SOM wraz z wykorzystywanymi narzędziami programistycznymi. Ponadto, zawiera opis parametrów sieci SOM z zasadami doboru.

- **Rozdział 5: Przeprowadzone badania.**
  Przybliża opis metodologii badawczej oraz wyniki przeprowadzonych badań. Każdy podrozdział zawiera szczegółowe omówienie wyników dla poszczególnych zbiorów danych oraz przedstawia wnioski.

- **Rozdział 6: Podsumowanie.**
  Zawiera wnioski z przeprowadzonych badań oraz propozycje potencjalnych kierunków dalszych badań w omawianej tematyce.

2. TEORETYCZNY OPIS SZTUCZNYCH SIECI NEURONOWYCH

Sztuczne sieci neuronowe (ang. *Artificial Neural Networks*, skrót *ANN*) , potocznie zwane sieciami neuronowymi, stanowią aktualnie jedne z najprężniej rozwijających się systemów przetwarzania informacji. Koncept ich rozwiązania bazuje na mechanizmach zachodzących w ludzkim mózgu, który bez wątpienia można nazwać najważniejszym organem w organizmie człowieka. Model sztucznego neuronu, czyli podstawowej jednostki obliczeniowej w sieciach neuronowych, inspirowany jest komórką nerwową, zwaną również neuronem [1]. Odpowiednie połączenia między neuronami były inspirowane budową biologicznych sieci neuronowych, występujących w ludzkim systemie nerwowym.

Sztuczne sieci neuronowe cieszą się rosnącą popularnością ze względu na swoją wszechstronność. Są narzędziem uniwersalnym, szeroko stosowanym, zarówno w środowisku akademickim jak i w przemyśle [2]. Ich wyjątkową cechą jest zdolność samodzielnego dostosowywania parametrów w procesie uczenia (tak zwanych wag), co umożliwia poprawne reagowanie na dane wejściowe w fazie pracy, nawet dla pobudzeń, które nie wystąpiły w fazie treningu [1, 3].

Na początku prezentowanego rozdziału przedstawiony zostanie model sztucznego neuronu, zawierający również krótki opis biologicznej komórki nerwowej. W kolejnych częściach omawiane będą takie zagadnienia jak wybrane struktury sieci neuronowych oraz proces uczenia wraz z podziałem.

2.1. Opis sztucznego neuronu

W celu lepszego zrozumienia zasady działania sztucznego neuronu przedstawione zostaną w skrócie zasady działające w systemie nerwowym człowieka, na którym bazuje koncept sztucznych sieci neuronowych.

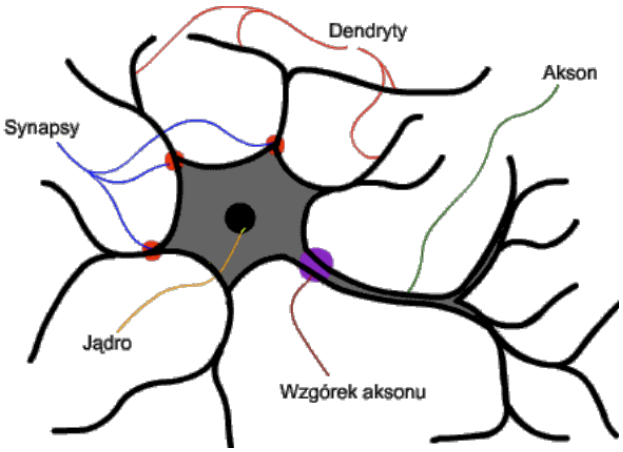
Neuron biologiczny

Jednym z elementów, z jakich składa się mózg ludzki, są komórki nerwowe, zwane neuronami. Ich ilość w mózgu można oszacować na około 10^{11} [2]. Komórki te są podstawowymi elementami, tworzącymi biologiczną sieć neuronową [4, 1]
Uproszczoną budowę neuronu można przedstawić jako [1]:

- Ciało komórki (ang. *soma*) wraz z jądrem komórkowym (ang. *nucleus*) - jest to centrum neuronu, w którym odbywa się przetwarzanie informacji. Oprócz tego zawiera jądro komórkowe oraz większość organelli, które odpowiadają za prawidłowe funkcjonowanie komórki nerwowej.
- Dendryty (ang. *dendrite*) - rozgałęzione wypustki neuronu, które odbierają sygnały wejściowe z innych komórek i przekazują je do ciała komórki.
- Akson (ang. *axon*) - długa wypustka neuronu, która przewodzi wyjściowe impulsy nerwowe z ciała komórki do dendrytów innych komórek.

Komórka nerwowa zawiera oczywiście dużo więcej elementów, jednak na potrzeby pracy wystarczy przytoczenie głównych składników.
Neuron działa poprzez odbieranie sygnałów elektrycznych od innych komórek za pomocą dendrytów, które następnie przekazują je do ciała komórki. W ciele komórki sygnały są sumowane

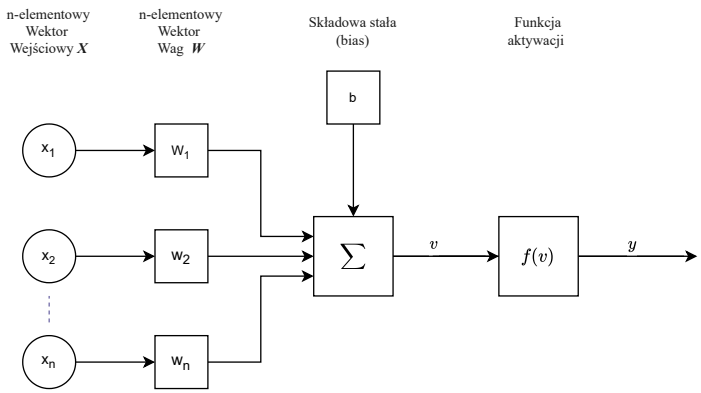
w krótkim przedziale czasowym, zwanym okresem utajonego sumowani [3]. Ciało komórki tworzy impuls wyjściowy, jeśli skumulowany potencjał impulsów przekroczy określony próg aktywacji, przy czym sygnały wejściowe dochodzące do ciała komórki mogą być hamujące lub pobudzające, co można rozumieć jako dodatnie bądź ujemne. Wytworzony sygnał jest przekazywany następnie do kolejnych komórek nerwowych za pomocą aksonu [1].
Schemat biologicznego neuronu przedstawiono na rysunku 2.1 [4].



Rysunek 2.1: Schemat budowy komórki nerwowej, z pracy [4]

Model sztucznego neuronu

Klasyczny model sztucznego neuron bazuje na uproszczonej zasadzie działania neuronu biologicznego, opisanej w sekcji 2.1. Sztuczny neuron posiada wiele wejść i jedno wyjście [1]. Na każde z wejść podać można wartość wyjścia innego neuronu, bądź bezpośrednio wartość tak zwanego wzorca wejściowego , rozumianego jako zewnętrzną wartość pobudzenia podanego na sieć w celu przetworzenia. Oprócz tego model sztucznego neuronu zawiera blok sumacyjny oraz blok wyznaczający wartość funkcji aktywacji [2]. Wszystkie z tych elementów zostaną omówione w dalszej części rozdziału.
Graficzny schemat sztucznego neuronu został zaprezentowany na rysunku 2.2 [1].



Rysunek 2.2: Schemat modelu sztucznego neuronu

Pierwszy krok przetwarzania informacji przez neuron stanowi sumowanie wartości poszczegól-

nych wejść przemnożonych przez wagi [5]. Pod pojęciem wagi rozumie się tutaj parametr przypisany każdemu wejściu, który określa, jak silny wpływ dane wejście ma na całkowity wynik. Do tej sumy niekiedy dodaje się czynnik stały, zwany również składową stałą (ang. *bias*) [1]. Podejście to można również zapisać wektorowo jako iloczyn, transponowanego wektora wag z wektorem wejściowym.
Matematycznie etap ten można opisać zależnościami 2.1 [3]

$$v = \mathbf{W}^{\mathsf{T}} \cdot \mathbf{X} + b = \sum_{i=1}^{n} w_i x_i + b \tag{2.1}$$

gdzie v to całkowita wartość pobudzenia neuronu, **W** to wektor wag, **X** to wektor wejściowy, b to bias (składowa stała). x_i oznacza natomiast *i*-te wejście, a w_i wagę przypisaną *i*-temu wejściu.

W kolejnym kroku następuje wyznaczenie wartości wyjściowej neuronu y, rozumianej jako wartość funkcji aktywacji $f()$ na podstawie całkowitej wartości pobudzenia neuronu v.
Zależność tą przedstawiono na 2.2 [4]

$$y = f(v) \tag{2.2}$$

Wybór funkcji aktywacji neuronu jest niezwykle istotny w kontekście przyszłego działania sieci. Sieci neuronowe można podzielić na liniowe i nieliniowe [2], różniące się między sobą zastosowaną funkcją aktywacji. Przy użyciu nieliniowych funkcji aktywacji łatwiej odwzorowywać dane o nieregularnych strukturach. Podejście nieliniowe bliższe jest naturalnym procesom zachodzącym w biologicznych sieciach neuronowych, ponieważ one również nie działają w sposób liniowy. Ze względu na większą użyteczność sieci z nieliniowymi funkcjami aktywacji [1], ich funkcję zostaną poniżej pokrótce zaprezentowane.
W literaturze zaproponowano wiele różnych nieliniowych funkcji aktywacji, począwszy od prostych funkcji takich jak funkcje progowe, aż po bardziej złożone, takie jak funkcja sigmoidalna [1, 2]. Każda z nich znajduje zastosowanie w innych typach problemów.

W początkowym okresie prac nad rozwojem sztucznych sieci neuronowych stosowano głównie funkcje progowe, przedstawione zależnościami 2.3 oraz 2.4 [3]. Ze względu na ich nieciągłość i gwałtowną zmianę sygnału wyjściowego nie są jednak tak powszechnie używane jak funkcja sigmoidalna, przedstawiona za pomocą wyrażenia 2.5 [1]. W zależności tej występuje parametr β, który jest parametrem odpowiadającym za kształt charakterystyki funkcji.

$$f(v) = \begin{cases} 1 & \text{dla } v \geq 0 \\ 0 & \text{dla } v < 0 \end{cases} \tag{2.3}$$

$$f(v) = \begin{cases} 1 & \text{dla } v \geq 0 \\ -1 & \text{dla } v < 0 \end{cases} \tag{2.4}$$

$$f(v) = \frac{1}{1 + \exp(-\beta \cdot v)} \tag{2.5}$$

Idea neuronu sztucznego jest uproszczonym modelem neuronu biologicznego. Zastosowanie funkcji nieliniowych pozwala na osiągnięcie lepszych rezultatów [1] w przedstawionych zadaniach. W kolejnej sekcji tego rozdziału przedstawione zostaną sposoby grupowania neuronów w sieć, w celu realizacji postawionych zadań.

2.2. Klasyczny model sztucznych sieci neuronowych

Zwięzłe zdefiniowanie, w jednym lub kilku zdaniach, tak obszernego zagadnienia, jakim są sztuczne sieci neuronowe jest zadaniem bardzo trudnym. W literaturze spotyka się różne definicje, bazujące na tym samym koncepcie. Na potrzeby tej pracy zostanie użyta ogólna definicja zaprezentowana w książce [1]:
”Sieć neuronowa jest układem neuronów, w którym wyjścia każdego neuronu są połączone, poprzez wagi, z wejściami wszystkich neuronów, w tym także z jego własnym wejściem”.
Z przytoczonej definicji jasno wynika, że sieć składa się z wielu neuronów. Możliwości pojedynczego sztucznego neuronu są dość ograniczone, podobnie jak ma to miejsce w ich biologicznych odpowiednikach [5]. Do celów lepszej realizacji założonych zadań neurony grupuje się w warstwy, rozumiane jako zbiory neuronów działających równocześnie i przetwarzających te same dane wejściowe [3].

Sztuczna sieć neuronowa może zawierać wiele warstw, połączonych ze sobą w taki sposób, że wyjście jednej z warstw jest wejściem kolejnej. Warstwy mogą różnić się między sobą ilością neuronów oraz pełnić zróżnicowane role, w zależności od konkretnego zastosowania sieci [1]. W większości współcześnie stosowanych sieci neuronowych warstwy dzieli się na: wejściową, na której wejścia podaję się wektory wejściowe, wyjściową, z której wyjścia stanowią odpowiedź sieci na dane pobudzenie oraz warstwy ukryte, znajdujące się między wyjściem neuronów z warstwy wejściowej oraz wejściem neuronów w warstwie wyjściowej [2].

Działanie sieci neuronowej można opisać w dwóch fazach [3]. Pierwsza z nich to faza treningu, nazywana też fazą uczenia. Polega na przekazywaniu do sieci wzorców uczących ze zbioru treningowego i obserwowaniu reakcji sieci. W procesie tym, w sposób określony danym algorytmem uczenia, sieć dostraja swoje wagi, w celu poprawy działania modelu [1]. Więcej informacji o procesie uczenia sieci zawarte zostanie w sekcji 2.3.
Drugim etapem jest faza właściwego działania sieci neuronowej, w której po przekazaniu wzorca wejściowego, sieć samodzielnie dokonuje przetwarzania informacji i podaje swoją odpowiedź. Ta zdolność sieci nie byłaby możliwa bez etapu treningu, ponieważ to wagi poszczególnych neuronów reprezentują pamięć i wiedzę nabytą w procesie uczenia [5].

Z uwagi na to, że sieć neuronowa do swojego poprawnego działania wymaga fazy treningu, w której sieć samodzielnie dostosowuje swoje wagi, zadaniem jej projektanta jest przede wszystkim odpowiedni dobór parametrów sieci, takich jak na przykład jej architektura, rozumiana jako liczba neuronów w poszczególnych warstwach, liczba warstw [1], poprawna implementacja algorytmu uczącego oraz wybór odpowiedniego zbioru uczącego.

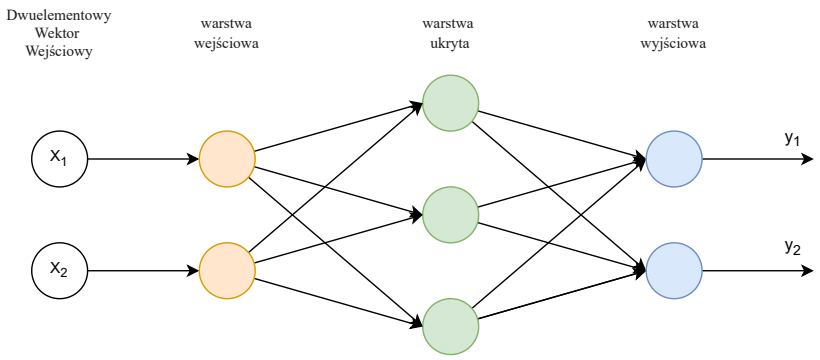
Podziału sieci neuronowych można dokonać względem wielu kryteriów, szerzej opisanych w takich pozycjach jak [5, 2, 1]. W tej pracy omówiony zostaną podziały ze względu na zastosowaną architekturę oraz na rodzaj algorytmu ucznia. W kolejnej sekcji przedstawiony zostanie podział względem stosowanej architektury.

Podział sieci neuronowych względem architektury

Ze względu na zastosowaną architekturę sieci można podzielić na wiele rożnych grup, z pośród których podstawowe to:

- sieci jednokierunkowe (ang. *Feedforward Neural Networks*, skrót FNN)
- sieci rekurencyjne (ang. *Recurrent Neural Networks*, skrót RNN), nazywane również sieciami ze sprzężeniem zwrotnym

Nazwa ”jednokierunkowe” związana jest ze sposobem w jaki sieć przesyła sygnały w swoim obrębie - zawsze w kierunku od warstwy wejściowej, przez warstwy ukryte, aż do warstwy wyjściowej. Ten mechanizm można również nazwać brakiem sprzężenia zwrotnego [3].
Sieci FNN, w większości spotykanych zastosowań, są zazwyczaj wielowarstwowe, czyli zawierają warstwy ukryte. Zdarzają się jednak sieci jednowarstwowe, których przykładami mogą być między innymi sieci Kohonena, opisane w tej pracy w rozdziale 3.
Schematyczny obraz przepływu informacji w sieciach FNN, dla przykładu sieci z wektorem wejściowym dwuelementowym oraz dwoma wyjściami, przedstawiony został na rysunku 2.3 [1].

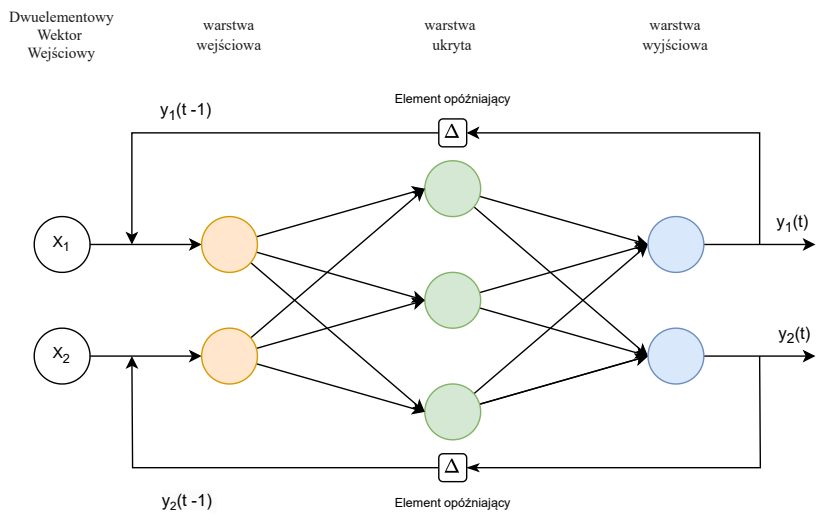


Rysunek 2.3: Schemat przepływu informacji w sieciach FNN, [1]

Rekurencyjne sieci neuronowe posiadają sprzężenie zwrotne, czyli mechanizm, w którym wyjścia neuronów z warstw bliżej warstwy wyjściowej mogą być ponownie wprowadzane na wejście sieci [1]. Mechanizm ten powoduje, że sieć staje się dynamiczna, czyli jej odpowiedź zależy nie tylko od podanego wzorca wejściowego, ale też od stanu początkowego sieci, na który wpływ miały poprzednie pobudzenia. Modyfikacja ta często daje możliwość przetwarzania skomplikowanych wzorców z wykorzystaniem mniejszej ilości neuronów czy ich warstw, w porównaniu do sieci FNN [1].

Schematyczny obraz przepływu informacji w sieciach RNN, dla przykładu sieci z wektorem wejściowym dwuelementowym oraz dwoma wyjściami, przedstawiony został na rysunku 2.4. Na schemacie tym występują bloki z oznaczeniem Δ. Symbolizują one graficznie koncept opóźnienia wyjścia w celu podania go w następnym kroku na wejście sieci [1].

Cechą wspólną sieci jednokierunkowych i rekurencyjnych jest stała struktura sieci, utrzymana zarówno w procesie treningu, jak i właściwej pracy sieci. Oznacza to, że architektura zaproponowana przez użytkownika na etapie projektowania sieci, jest niezmienna i niemodyfikowalna [5].
Znane są rozwiązania sieci neuronowych, które same modyfikują swoją architekturę w procesie treningu. Takie sieci nazywa się sieciami ontogenicznymi. Zmiana rozkładu sieci może odbywać się poprzez ograniczanie lub zwiększanie liczby neuronów w poszczególnych warstwach, a czasem nawet poprzez usuwanie lub dodawanie całych warstw. Mechanizm ten pozwala na rozwiązanie problemu dobrania optymalnej architektury sieci,tak aby spełniała swoje zadania, równocześnie gwarantując minimalne zużycie zasobów. W sieciach jednokierunkowych i rekurencyjnych jest to zadanie użytkownika, z kolei w sieciach ontogenicznych początkowa architektura jest modyfikowana przez algorytm uczenia.
Powyżej opisane założenia sieci ontogenicznych są bardzo zwięzłe. Ich tematyka jest jednak bar-



Rysunek 2.4: Schemat przepływu informacji w sieciach RNN, [3]

dziej złożona i wymaga dogłębnego zrozumienia. W literaturze można znaleźć wiele szczegółowych opisów, między innymi w pozycji [6].

2.3. Uczenie sieci neuronowych

Faza uczenia sieci neuronowych jest niezbędna do wykonywania zadań założonych przez użytkownika. W literaturze wielokrotnie określana jest mianem najważniejszego etapu, w kontekście wpływu na końcową efektywność działania sieci neuronowej [1]. Szczególnym wyróżnikiem jest pełna autonomiczność procesu uczenia, niewymagająca od użytkownika bezpośredniej ingerencji w czasie jego trwania. Podczas uczenia sieć dostosowuje swoje wagi w oparciu o analizę dostarczonych danych, zgrupowanych w zbiorze uczącym. Skala dostosowania zależna jest od zastosowanego algorytmu uczenia [2].
Proces uczenia inspirowany jest naturalnym procesem poszerzania swojej wiedzy przez człowieka. W organizmie ludzkim nabieranie nowych umiejętności wymaga powtarzania pewnego procesu, na który składa się wykonywanie określonej czynności, obserwacji jej skutków i wyciągnięcie z nich wniosków. Wielokrotna realizacja tego cyklu powoduje stopniowe nabieranie umiejętności, aż do momentu, w którym można uznać, że dana czynność została opanowana. Zależność tą również jest obecna w procesie uczenia sztucznych sieci neuronowych. W tym wypadku również wymagane jest wielokrotne przedstawienie pewnego wzorca uczącego, wykonanie określonej czynności, zdefiniowanej przez projektanta, oraz określenie czy czynność ta została wykonana poprawnie. Wymagane jest wielokrotne powtórzenie procesu z zasilaniem go wzorcami wejściowymi z tego samego zbioru danych uczących [5].

Sposób reakcji sieci na wzorzec wejściowy, wraz z zakresem dostrajania wag, zależy od zastosowanej metody uczenia, zwanej również algorytmem uczenia. W ostatnich latach pojawiają kolejne propozycje jego implementacji, które jednak można podzielić na kategorię różniące się formą prezentacji wzorca wejściowego.
Zwykle wyróżnia się trzy podstawowe podziały algorytmów uczenia sieci neuronowych [3]:

- Uczenie z nauczycielem (ang. *Supervised Learning*)

- Uczenie bez nauczycielem (ang. *Unsupervised Learning*)
- Uczenie z krytykiem (ang. *Reinforcement Learning*)

W dalszych sekcjach tego rozdziału metody te zostaną opisane w sposób skondensowany.

Uczenie z nauczycielem

Uczenie z nauczycielem, znane jako uczenie nadzorowane, swoją nazwę zawdzięcza obecności umownego „nauczyciela", który podpowiada oczekiwane reakcje sieci neuronowej na dane pobudzenie [1]. Realizowane jest to w ten sposób, że podczas każdej iteracji, w fazie uczenia, na wejście sieci podaje się daną wejściową oraz oczekiwaną reakcję sieci, zwaną etykietą. Po przetworzeniu wzorca wejściowego przez sieć wyjście sieci porównuje się z etykietą i na tej podstawie modyfikowane są wagi neuronów [3]. Uczenie z nauczycielem jest jedną z najpopularniejszych metod uczenia sztucznych sieci neuronowych i stanowi bardzo obszerne zagadnienie. Na jakość modelu wpływ ma między innymi wybrana reguła modyfikacji wag (jak na przykład reguła delta czy algorytm wstecznej propagacji błędu (BP)), rozmaite optymalizatory oraz szereg innych parametrów. Dobór tych parametrów został szeroko opisany w literaturze, w pozycjach [2, 1, 7, 5].

Uczenie bez nauczyciela

Uczenie bez nauczyciela, nazywane również uczeniem nienadzorowanym, polega na przekazywaniu na wejście sieci neuronowej jedynie danych wejściowych. Konsekwencją tego jest brak znajomości poprawnej reakcji sieci. W efekcie sieć sama uczy się rozpoznawać wzorce w danych wejściowych, na przykład na podstawie ich powtarzalności, podobieństw między poszczególnymi elementami czy charakterystycznych cech grupujących. [2]. Proces ten często nazywany jest samouczeniem lub samoorganizacją [1].
Za podstawowe metody uczenia nienadzorowanego uznaje się metody uczenia korelacyjnego, nazywane również metodą Hebba oraz metody rywalizacyjne, nazywane niekiedy uczeniem z regułami Kohonena [2]. Metody rywalizacyjne nazywane są czasem również metodami konkurencyjnymi. Obie z tych metod są szeroko opisywane w literaturze, między innymi w pozycjach [4, 1, 3]. Uczenie konkurencyjne, z zastosowaniem algorytmu "zwycięzca bierze większość" jest głównym tematem niniejszej pracy, z tego powodu zostanie on szczegółowo opisane w rozdziale 3.

Uczenie z krytykiem

Uczenie z krytykiem nazywa się często uczeniem ze wzmocnieniem. Traktuje się je jako szczególną odmianę uczenia z nauczycielem. W tym wypadku nauczyciel nie wskazuje bezpośrednio oczekiwanych reakcji sieci, a jedynie informację, czy podjęta przez sieć decyzja zbliża ją do oczekiwanego rezultatu czy oddala. Gdy reakcja zbliża system do osiągnięcia celu założonego przez projektanta, przyznawana jest metaforyczna „nagroda", rozumiana jako zachęta do kontynuacji tej strategii. W przeciwnym razie przyznaje się "karę", która jest sygnałem dla sieci, że poszukuje się innych wzorców [3]. W związku z tym, jako różnicę między klasycznym uczeniem z nauczycielem, a uczeniem z krytykiem, można nakreślić moment podania informacji o poprawności podjętych przez sieć działań. W uczeniu nadzorowanym sieć zawsze otrzyma wartość referencyjną, która jest stała i niezależna od odpowiedzi sieci. W uczeniu ze wzmocnieniem natomiast kara bądź nagroda przyznawana jest dopiero po wykonaniu przetwarzania przez sieć i podjęciu działań [3].

2.4. Podsumowanie rozdziału

Zaprezentowany rozdział miał na celu przybliżenie czytelnikowi ogólnych wiadomości z zakresu tematyki sztucznych sieci neuronowych. Opis został celowo skrócony, koncentrując się wyłącznie na kluczowych pojęciach, które są niezbędne dla zrozumienia kolejnych rozdziałów. Pominięto szczegółowe omówienie takich aspektów jak powszechnie stosowane typy sieci, praktyczne zastosowania sieci neuronowych czy reguły ich uczenia.

Z uwagi na to, że głównym tematem pracy są sieci samoorganizujące, korzystające z algorytmu ”zwycięzca bierze większość”, rozdział 3 dotyczył będzie opisu tych właśnie sieci.

3. SAMOORGANIZUJĄCE SIĘ SZTUCZNE SIECI NEURONOWE

Samoorganizujące się sztuczne sieci neuronowe, znane również jako mapy SOM (ang. *Self-Organizing Map*), sieci neuronowe Kohonena lub sieci SOFM (ang. *Self-Organizing Feature Map*), stanowią przykład sieci neuronowych uczonych bez nauczyciela (ang. *unsupervised learning)* [4]. Zostały one zaprezentowane w 1988 roku przez fińskiego profesora Teuvo Kohonena [1, 4, 8]. W oryginalnej koncepcji zaproponowano dwie metody uczenia: algorytm „zwycięzca bierze wszystko" (ang. *Winner Takes All*, skrót WTA) oraz algorytm „zwycięzca bierze większość" (ang. *Winner Takes Most*, skrót WTM) [8].

Sposób działania sieci neuronowych Kohonena opiera się na procesie samoorganizacji, występującym także w ludzkim mózgu. W systemie nerwowym człowieka różne narządy przekazują informacje do mózgu za pośrednictwem układu nerwowego, gdzie dochodzi do odwzorowywania i zestawiania informacji pochodzących ze zmysłów oraz porównywania ich z wcześniej nabytymi wzorcami zachowań. Reguły te można traktować jako źródło inspiracji do opracowania sieci SOM.

Dzięki zastosowaniu techniki uczenia bez nadzoru, sieci SOM stają się wyjątkowo użyteczne w analizie i klasyfikacji dużych zbiorów danych, w których trudne jest przewidzenie struktury, rozumianej jako podział danych na grupy,ich liczność oraz powtarzalność. Zdolność sieci Kohonena do odwzorowywania wielowymiarowych przestrzeni danych na prostsze struktury znajduje zastosowanie w rozwiązywaniu złożonych problemów [9, 3, 1].

Mapy SOM mają na celu odwzorowanie wielowymiarowych danych wejściowych na przestrzenie o mniejszej liczbie wymiarów, zazwyczaj jedno- lub dwuwymiarowe. Zadanie to wymaga odpowiedniego sklasyfikowania wektorów wejściowych, według określonych kryteriów podobieństwa, przy jednoczesnym zachowaniu istotnych cech danych. W procesie uczenia, sieć SOFM grupuje wzorce uczące w klasy na podstawie podobieństwa sygnałów i przypisuje danej klasie neuron wyjściowy [8, 4].

Samoorganizujące się sieci neuronowe określa się jako sieci z uczeniem konkurencyjnym (ang. *competitive learning*). Uczenie konkurencyjne polega na ocenie podobieństwa wektora wejściowego z wartościami wag neuronów. Konkurencyjność tego rozwiązania oznacza, że tylko neuron o najbardziej zbliżonych wagach (lub wąska grupa neuronów, jak w algorytmie WTM oraz w modelach z algorytmem gazu neuronowego ma prawo adaptacji swoich wag, tak aby lepiej reprezentować wzorzec wejściowy [1, 3].

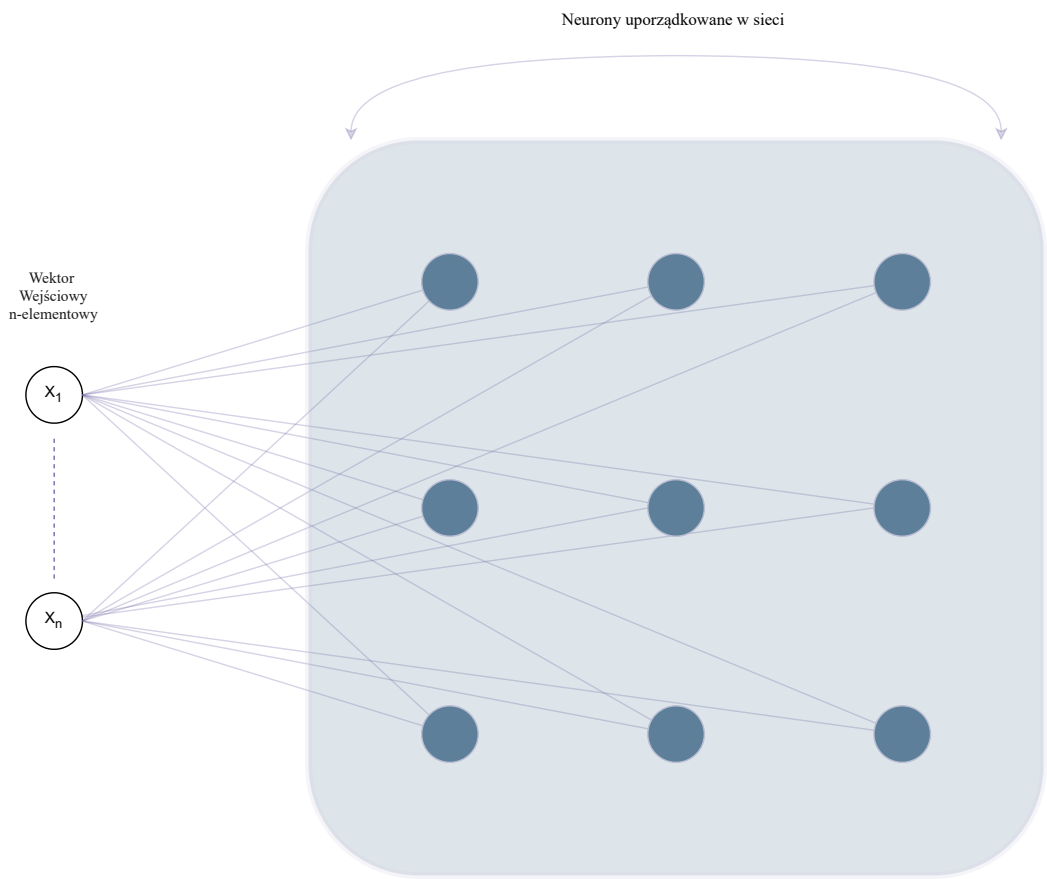
Niniejsza praca poświęcona jest sieciom neuronowym Kohonena z algorytmem „zwycięzca bierze większość" (ang. *Winner Takes Most*, WTM). W kolejnych częściach rozdziału omówione zostaną najpopularniejsze metody uczenia sieci SOM, ze szczególnym uwzględnieniem algorytmu WTM. Ponadto zaprezentowane zostaną również inne metody uczenia sieci SOM, takie jak algorytm WTA i gazu neuronowego. Następnie omówione zostaną miary jakości wytrenowania modelu oraz wybrane zastosowania sieci samoorganizujących się, ze szczególnym naciskiem na ich wykorzystanie w zadaniach klasyfikacji i klasteryzacji.

3.1. Opis sieci samoorganizujących się

Samoorganizujące się sieci neuronowe to jednowarstwowe sieci, w których każdy *n*-wymiarowy wektor wejściowy połączony jest z każdym neuronem. Sieć można przedstawić jako siatkę, bądź mapę, z liczbą ”punktów”równą liczbie neuronów w sieci [4]. Rysunek takiej siatki przedstawiono na rysunku rys. 3.1, dla wektora wejściowego o rozmiarze *n* i sieci składającej się z 9 neuronów uporządkowanych w siatkę o topologii prostokątnej 3x3.

Trening sieci samoorganizującej polega na wielokrotnym dostosowywaniu wag neuronów, aby odzwierciedlały one strukturę danych wejściowych. Sieć uczy się rozpoznawać podobieństwa między wzorcami, stopniowo organizując neurony w sposób, który odpowiada relacjom obecnym w zbiorze danych.

W wyniku procesu treningu sieci, wagi neuronów zostają dostosowane w taki sposób, aby każdy neuron reprezentował jedną z klas ze zbioru wejściowego, odpowiadając najbardziej charakterystycznym wzorcom obecnym w danych. O jakości wytrenowania sieci świadczą wartości miar oceny efektywności sieci, opisane szerzej w sekcji 3.5.



Rysunek 3.1: Architektura sieci SOM

Algorytmy uczenia sieci SOM

Algorytm uczenia zastosowany w sieci samoorganizującej odgrywa kluczową rolę, ponieważ jego wybór wpływa na jakość końcowych rezultatów, czas potrzebny na trening oraz zużycie zasobów

obliczeniowych. Różne algorytmy mogą znacząco różnić się efektywnością, zwłaszcza w odniesieniu do złożoności danych i precyzji odwzorowania. Odpowiednie dobranie metody uczenia jest zatem niezbędne do uzyskania optymalnych wyników przy jednoczesnej minimalizacji kosztów obliczeniowych, co pokazano w takich publikacjach jak [3, 9].

W dalszej części pracy zaprezentowane zostaną następujące algorytmy uczenia stosowane w sieciach SOM:

- algorytm **zwycięzca bierze wszystko, WTA** (ang. *Winner Takes All*),
- algorytm **zwycięzca bierze większość, WTM** (ang. *Winner Takes Most*),
- algorytm **gazu neuronowego, NG** (ang. *Neural Gas*).

Należy dodać, że algorytmów uczenia sieci samoorganizujących jest znaczenie więcej, jednak w ramach niniejszej pracy opisane zostaną tylko wyżej wymienione. Szerszy opis można znaleźć w pozycjach [3, 4].

3.2. Algorytm Zwycięzca bierze większość (WTM)

Algorytm WTM jest, obok algorytmu WTA, jednym z najpowszechniej stosowanych algorytmów w uczeniu sieci SOM.

Algorytm WTM oparty jest na rywalizacji między neuronami. Po przekazaniu do sieci *n*-wymiarowego wektora wejściowego X wybierany jest neuron o wagach najbardziej zbliżonych wartości X, który następnie, wraz z wybranymi neuronami sąsiadującymi, aktualizuje swoje wagi, aby lepiej reprezentować dany wzorzec. [9, 4, 10]

Niekiedy w literaturze algorytm WTM dzieli się na fazę rywalizacyjną (ang. *Competitive Phase*) oraz fazę kooperacyjną (ang. *Cooperative Phase*) [10, 7], co pozwala podkreślić zarówno konkurencyjny charakter wyboru neuronu ”zwycięzcy”, jak i współpracę neuronów sąsiednich podczas aktualizacji wag.

Niniejsza praca poświęcona jest wykorzystaniu algorytmu WTM oraz prezentacji procesu uczenia dla różnych przypadków budowy siatki, jak i definicji funkcji sąsiedztwa.

3.2.1. Opis algorytmu

W pierwszym kroku algorytmu przypisuje się losowe wartości wag dla każdego neuronu. Przy inicjalizacji istotną kwestią jest możliwe równomierne rozłożenie neuronów, tak aby w początkowych etapach treningu neurony mogły reprezentować poprawnie dane wejściowe [1].

W kolejnym kroku działania sieci oblicza się odległość $D(X, W_j)$ między wektorem wejściowym, a wektorem wag każdego neuronu w sieci neuronowej. Wybór metryki, w której mierzona jest odległość między wektorem wejściowym a wagami neuronów, ma wpływ na końcowy wynik (w tym efektywność) uczenia sieci.

Najczęściej używane miary [3]:

- miara euklidesowa (według normy L2)

$$D(X, W_j) = \sqrt{\sum_{c=1}^{n} (x_{c,c} - w_{j,c})^2}, \tag{3.1}$$

- miara według normy L1 (tzw. miara Manhattan)

$$D(X, W_j) = \sum_{c=1}^{n} |x_{c,c} - w_{j,c}|, \tag{3.2}$$

- iloczyn skalarny (tzw. miarta kosinusowa)

$$D(X, W_j) = X \cdot W_j = \|X\|\|W_j\| \cos(X, W_j), \tag{3.3}$$

- miara według normy L_∞

$$D(X, W_j) = \max_c |x_j - w_j|, \tag{3.4}$$

gdzie X to *n*-wymiarowy wektor wejściowy, W_j – *n*-wymiarowy wektor wag j-tego neuronu , $D(X, W_j)$ – odległość pomiędzy wektorem wejściowym X a wektorem wag W_j j-tego neuronu. Spośród wyżej wymienionych miar podobieństwa najczęściej używana jest miara Euklidesa, chociaż patrząc z punktu widzenia szybkości działania lepsza wydaje się i jest miara według normy L1.

Po procesie wyznaczania odległości $D(X, W_j)$ zostaje wybrany neuron najbardziej podobny do wzorca wejściowego – tak zwany "zwycięzca"(ang. *"winner"*). W momencie wyznaczenia neuronu zwycięzcy kończy się faza rywalizacji (rozumiana jako wybór neuronu zwycięzcy na podstawie zadanej miary podobieństwa), a rozpoczyna faza kooperacji, podczas której neuron zwycięzca oraz neurony należące do jego sąsiedztwa (ang. *"neighbors"*) aktualizują swoje wagi [7, 10]. Dostosowywanie wag przebiega zgodnie z zależnością [2]:

$$W_j(l+1) = W_j(l) + \eta(k) \cdot G(R, d(i,j)) \cdot [X(l) - W_j(l)], \tag{3.5}$$

gdzie $W_j(l)$ to wektor wag neuronu w iteracji l, $\eta(k)$ oznacza współczynnik uczenia w epoce k, $G(R, d(i,j))$ to funkcja sąsiedztwa, zależna od promienia R oraz odległości topologicznej $d(i,j)$ pomiędzy neuronami i i j, $X(l)$ to wektor wejściowy w iteracji l.

W zależności (3.5) pojawiają się niespotykane dotąd pojęcia takie jak funkcja sąsiedztwa $G(R, d(i,j))$, promień sąsiedztwa R oraz odległość topologiczna między neuronami $d(i,j)$. Ostatnie dwa z tych pojęć związane są z pojęciem topologii sieci. Terminy te zostaną wyjaśnione w kolejnych sekcjach tego rozdziału.

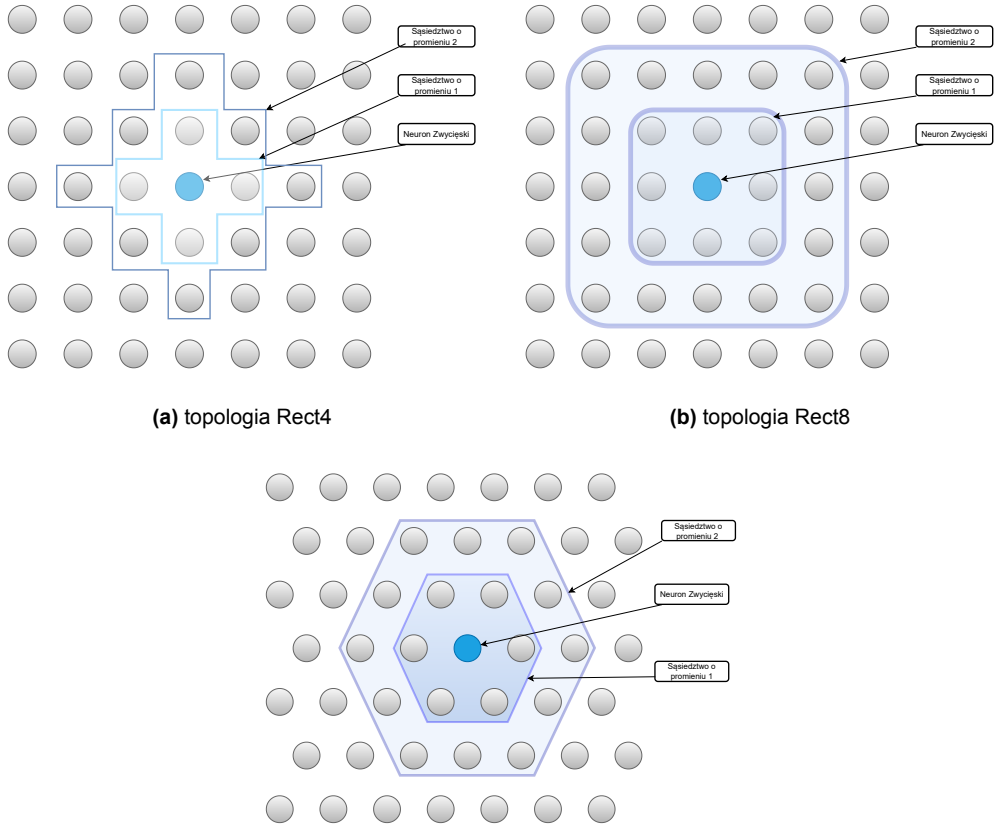
3.2.2. Topologia samoorganizującej się sieci neuronowej

Pojęcie topologii sąsiedztwa w sieciach SOM odnosi się do przestrzennej organizacji neuronów. Neurony są zorganizowane w regularną siatkę, zwykle jedno- lub dwuwymiarową, gdzie każdy neuron ma określony zestaw sąsiadów, zależny od promienia sąsiedztwa w danej iteracji. Wyjątkiem tu mogą być neurony leżące na krawędziach siatki, które siłą rzeczy będą posiadały mniejszą liczbę sąsiadów [4].

Topologia sąsiedztwa definiuje, które neurony są traktowane jako bliskie sobie w sensie odległości topologicznej, a to bezpośrednio wpływa na proces uczenia. W miarę postępu procesu uczenia sieci, neurony znajdujące się bliżej siebie, w zadanej topologii, będą bardziej skłonne do aktualizowania swoich wag w podobny sposób, co pozwali na zachowanie relacji przestrzennych pomiędzy wzorcami danych. W typowych przypadkach stosuje się topologię prostokątną lub heksagonalną [8], których obraz przedstawiono na rysunku (3.2).

Wartość promienia sąsiedztwa w sieci SOM maleje stopniowo w trakcie procesu uczenia, przechodząc od wartości maksymalnej na początku treningu, do zera pod koniec. [8] Na początku, gdy promień sąsiedztwa jest duży, dostosowywanie wag obejmuje większą liczbę neuronów wokół neuronu „zwycięskiego", co umożliwia sieci szerokie uogólnienie i uchwycenie globalnych wzorców danych w pierwszych epokach treningu [4].

W miarę, jak wartość promienia maleje z każdą iteracją, obszar sąsiedztwa skupia się coraz bardziej wokół neuronu zwycięskiego, redukując liczbę neuronów, które dostosowują swoje wagi. Ostatecznie, gdy promień sąsiedztwa osiągnie zero, dostosowywany jest jedynie neuron „zwycięzca”, co sprawia, że sieć działa podobnie do algorytmu „zwycięzca bierze wszystko” (ang. *Winner Takes All*, WTA), [4] opisanego dalej w pracy w podrozdziale (3.3). W takim przypadku zmiany są bardziej lokalne i precyzyjne, co umożliwia sieci WTM dostrajanie swoich wag w końcowym etapie treningu [8].



(a) topologia Rect4

(b) topologia Rect8

(c) topologia hexagonalna

Rysunek 3.2: Przykłady topologi sieci SOM: a) topologia prostokątna z czterema sąsiadami (Rect4), b) topologia prostokątna z ośmioma sąsiadami (Rect8), c) topologia hexagonalna.

3.2.3. Funkcje sąsiedztwa

Funkcja sąsiedztwa w algorytmie WTM jest elementem wyróżniającym ten algorytm spośród innych, odgrywając kluczową rolę w procesie uczenia sieci. Funkcja ta decyduje o tym, które neurony będą poddane aktualizacji wag w odpowiedzi na wzorzec wejściowy oraz w jakim stopniu wagi te zostaną zmienione w zależności od odległości od neuronu zwycięskiego [4, 7].

Dobór odpowiedniej funkcji sąsiedztwa ma kluczowe znaczenie, ponieważ wpływa nie tylko na jakość wytrenowania sieci, ale także na koszty związane z treningiem modelu, takie jak zużycie mocy obliczeniowej. W niniejszej pracy sieci SOM implementowane są programowo, co umożliwia elastyczne testowanie różnych funkcji sąsiedztwa i ocenę ich efektów. W przypadku implementacji sprzętowych, gdzie zasoby obliczeniowe są ograniczone, wybór funkcji sąsiedztwa staje się jednak szczególnie istotny, ponieważ wymaga kompromisu między jakością modelu a

kosztami mocy obliczeniowej i czasem treningu [4, 11].

Najprostszą stosowaną funkcją sąsiedztwa jest prostokątna funkcja sąsiedztwa (ang. *Rectangular Neighborhood Function*), znana również jako funkcja *bubble*. Definiowana jest wzorem (3.6) [3] :

$$G(R, d(i,j)) = \begin{cases} 1 & \text{dla } d(i,j) \leq R, \\ 0 & \text{dla } d(i,j) > R, \end{cases} \tag{3.6}$$

gdzie $G(R, d(i,j))$ oznacza wartość funkcji sąsiedztwa, R jest promieniem sąsiedztwa, a $d(i,j)$ to odległość topologiczna między neuronem i a neuronem j. Prostokątną funkcję sąsiedztwa stosuje się ze względu na prostotę jej implementacji oraz niski koszt obliczeniowy. Funkcja przyjmuje wartość 1 dla neuronów znajdujących się w promieniu R wokół neuronu zwycięskiego oraz 0 dla neuronów poza tym zakresem. Takie uproszczenie umożliwia bezpośrednie określenie sąsiedztwa, bez konieczności stosowania złożonych operacji matematycznych, co znacznie usprawnia proces uczenia.

Ze względu na ograniczoną precyzję funkcji *bubble*, częściej stosowanym rozwiązaniem jest funkcja sąsiedztwa Gaussa (ang. *Gaussian Neighborhood Function*), która zapewnia bardziej "płynny spadek" wartości sąsiedztwa w zależności od odległości. Jest ona definiowana wzorem (3.7) [3]

$$G(R, d(i,j)) = \text{exp}\left(-\frac{d(i,j)^2}{2R^2}\right), \tag{3.7}$$

Funkcja Gaussa umożliwia bardziej płynne (stopnieowe) zmniejszanie wartości funkcji sąsiedztwa wraz ze wzrostem odległości od neuronu zwycięskiego. Zapewnia to bardziej stabilne niż poprzednio dopasowanie wag sąsiednich neuronów. Dzięki temu możemy także lepiej odwzorować zależności między wzorcami, co sprzyja naturalnemu kształtowaniu struktury mapy SOM.

Innymi często spotykanymi funkcjami są funkcja eksponencjalna [12], opisana zależnością (3.8) i funkcja trójkątna [13] opisana zależnością (3.9).

$$G(R, d(i,j)) = \begin{cases} 0.5^{d(i,j)} & \text{dla } d(i,j) \leq R, \\ 0 & \text{dla } d(i,j) > R \end{cases} \tag{3.8}$$

Funkcja eksponencjalna, podobnie jak gaussa, zapewnia stopniowe zmniejszanie wartości funkcji sąsiedztwa wraz ze wzrostem odległości d_{ij} między neuronami i i j.

$$G(R, d(i,j)) = \begin{cases} 1 - \frac{d(i,j)}{R} & \text{dla } d(i,j) \leq R, \\ 0 & \text{dla } d(i,j) > R \end{cases} \tag{3.9}$$

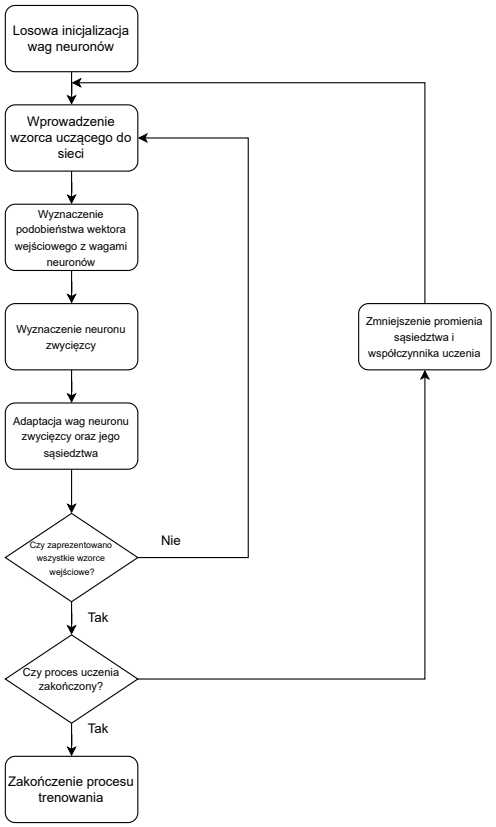
Funkcja trójkątna pozwala na liniowy spadek wartości sąsiedztwa w miarę zwiększania się odległości od neuronu zwycięskiego aż do zera poza zakresem promienia $R(t)$. Funkcja ta jest znacznym uproszczeniem funkcji Gaussa, z uwagi na brak konieczności stosowania skomplikowanych operacji arytmetycznych, dlatego często jest używana w implementacjach sprzętowych sieci SOM uczonych metodą WTM [4, 11].

W literaturze można spotkać także inne funkcje sąsiedztwa, które różnią się charakterystyką wpływu na sąsiednie neurony. Przykładem są funkcje sąsiedztwa typu *Cut Gaussian* oraz *Mexican-hat*, które wprowadzają różne modyfikacje do klasycznej funkcji Gaussa, wpływając na sposób, w jaki wartości sąsiedztwa maleją wraz z odległością od neuronu zwycięskiego. Te funkcje nie będą jednak przedmiotem niniejszej pracy [8, 13].

3.2.4. Adaptacja parametrów w czasie

W przedstawionym procesie uczenia algorytmu WTM istotne jest stopniowe zmniejszanie współczynnika szybkości uczenia η (ang. *learning rate*) oraz promienia sąsiedztwa R. Początkowo wysoka wartość η pozwala pojedynczemu neuronowi szybko dostosować się do głównych wzorców w danych, a jego stopniowe zmniejszanie umożliwia bardziej precyzyjne dopasowanie wag na dalszych etapach treningu [3]. Analogicznie, duża wartość promienia sąsiedztwa, na początku procesu uczenia pozwala na globalne uogólnienie danych poprzez aktualizacje wag wielu neuronów w sieci. W miarę postępu treningu promień maleje, zawężając obszar dostosowania do neuronów najbliższych neuronowi zwycięskiemu, co umożliwia dokładniejsze dostrajanie wag sieci [4].

Powyżej przedstawiony proces uczenia sieci metodą WTM został przedstawiony na schemacie blokowym, widocznym na rysunku (3.3)



Rysunek 3.3: Schemat blokowy algorytmu WTM

3.3. Algorytm WTA

Kolejną, po WTM, najczęściej stosowaną metodą uczenia sieci SOM jest algorytm „Zwycięzca bierze wszystko” (ang. *Winner Takes All*, skrót WTA). Został on zaproponowany przez Teuvo Kohonena w [8].

Ogólna struktura algorytmu WTA jest zbliżona do schematu przedstawionego wcześniej w algorytmie WTM, przedstawionego w sekcji (3.2)). Schemat działania WTA można przedstawić w kilku krokach:

1. losowa inicjalizacja wag neuronów,

2. obliczenie odległości pomiędzy wektorem wejściowym (uczącym) a wektorami wag wszystkich neuronów,

3. wybór neuronu zwycięskiego,

4. aktualizacja wag TYLKO dla neuronu zwycięskiego, zgodnie z ustalonym współczynnikiem szybkości uczenia, η.

Podstawową różnicą między algorytmem WTA i WTM jest brak fazy kooperacji. W algorytmie WTA tylko jeden neuron - zwycięski - dostosowuje swoje wagi do wektora uczącego, natomiast pozostałe neurony nie podlegają aktualizacji [7]. Pozostałe elementy procesu uczenia pozostają takie same, jak w algorytmie WTM – współczynnik uczenia stopniowo maleje w trakcie treningu, a odległość między wektorem wejściowym a wagami neuronów jest mierzona według jednej z wybranych metryk, opisanych w (3.2.1).

Brak funkcji sąsiedztwa w algorytmie WTA sprawia, że proces uczenia skupia się wyłącznie na dostosowaniu wag wybranego (zwycięskiego) neuronu, bez konieczności uwzględniania pozostałych (sąsiadujących). Takie uproszczenie eliminuje konieczność brania pod uwagę topologii sieci, ponieważ nie ma potrzeby wyznaczania wartości funkcji sąsiedztwa dla każdego neuronu w każdej iteracji. Dzięki temu algorytm WTA jest istotnie efektywniejszy obliczeniowo i mniej złożony w stosunku do WTM [1].

Uproszczenie to niesie ze sobą także pewne problemy. Jednym z nich jest zjawisko tzw. martwych neuronów, czyli neuronów, które nigdy nie wygrywają rywalizacji z innymi i pozostają nieaktywne w procesie uczenia. W literaturze spotyka się różne metody radzenia sobie z tym problemem, w tym zastosowanie mechanizmu „sumienia” *(ang. conscience mechanism*), który ma na celu eliminacje tego problemu [3, 14].

3.3.1. Mechanizm Sumienia

Mechanizm ”sumienia”, zwany również mechanizmem zmęczenia neuronów, polega na wprowadzeniu konieczności zapisu wyników rywalizacji, czyli zwycięskiego neuronu. Każdemu neuronowi przypisuje się dodatkowy parametr „potencjał”, która wzrasta, przy zwycięstwie neuronu i maleje w przeciwnym wypadku. W literaturze zaproponowano między innymi takie rozwiązanie mechanizmu sumienia [3]:

$$p_i(l+1) = \begin{cases} p_i(l) + \frac{1}{n} & \text{jeśli neuron } i \text{ jest zwycięski,} \\ p_i(l) - p_{\text{min}} & \text{jeśli neuron } i \text{ nie jest zwycięski} \end{cases} \tag{3.10}$$

gdzie $p_i(l)$ oznacza potencjał neuronu i w iteracji l, n reprezentuje całkowitą liczbę neuronów, p_{min} to minimalna potencjał uprawniający do wzięcia udziału w rywalizacji. Po wyznaczeniu wartości potencjału zwycięzcę wybiera się korzystając z zależności [3]:

$$d(x, W_w) = \text{min}\{d(x, W_i)\}, \quad 1 \leq i \leq n \quad \text{oraz} \quad p_i \geq p_{\text{min}} \tag{3.11}$$

Mechanizm ten pozwala ograniczyć liczbę martwych neuronów. Inne rozwiązania mechanizmu ”sumienia”zostały zaprezentowane również w [14]

3.4. Algorytm gazu Neuronowego

Ostatnim Algorytmem zaprezentowanym w pracy będzie algorytm gazu neuronowego (*ang. Neural Gas*). Rozwiązanie te zostało zaprezentowane w 1993 roku w pracy [15]. Jakkolwiek można się dopatrywać pewnych podobieństw względem metod WTM i WTA, w metodzie tej zastosowano pewne rozwiązania, które znacząco zmieniają proces uczenia sieci.

Uproszczony schemat algorytmu gazu neuronowego prezentuje się następująco:

1. Losowa inicjalizacja wag neuronów.
2. Obliczenie odległości pomiędzy wektorem wejściowym X a wektorami wag każdego z neuronów.
3. Sortowanie neuronów według rosnącej odległości od wzorca X.
4. Przypisanie indeksu h na podstawie jego pozycji w posortowanym zbiorze.
5. Aktualizacja wag neuronów.

Jak widać, pierwsze 2 kroki są takie same jak w algorytmach WTA i WTM. Po przedstawieniu wzorca uczącego X obliczana jest odległość między wektorem wag W a wektorem wejściowym X. Następnie wybierany jest neuron zwycięski, którego wagi są najbliższe, według wybranej miary podobieństwa Sekcja 3.2.1, wektorowi wejściowemu [15]. W przeciwieństwie do WTM i WTA, w algorytmie gazu neuronowego neurony są następnie sortowane według ich odległości od wzorca uczącego, a każdemu neuronowi przypisywany jest indeks h, określający jego miejsce w posortowanej kolejności [3]:

$$D(X, W_j)_0 < D(X, W_j)_1 < \cdots < D(X, W_j)_{n-1} \tag{3.12}$$

Po procesie sortowania neuronów, zwycięski neuron przyjmuje indeks $h = 0$, a kolejne neurony odpowiednio kolejne liczby całkowite ($h = 1, 2, \ldots$).

Modyfikacja wag w algorytmie *Neural Gas* odbywa się zgodnie z poniższą zależnością [15, 3]:

$$W_j(l+1) = W_j(l) + \eta(k) \cdot G(f) \cdot [X(l) - W_j(l)] \tag{3.13}$$

W algorytmie NG stosuje się funkcję sąsiedztwa $G(f)$, która obejmuje wszystkie neurony w sieci i maleje wraz ze wzrostem odległości wag neuronu od wzorca uczącego [15]. Funkcja ta jest opisana wzorem:

$$G(f) = \text{exp}\left(-\frac{h(j)}{\lambda}\right) \tag{3.14}$$

gdzie $h(j)$ określa pozycję j-tego neuronu w uszeregowanej liście neuronów, a λ jest parametrem odpowiadającym promieniowi sąsiedztwa R w WTM. Im mniejsza odległość między wektorem wejściowym, a wagami neuronu, tym silniejsza modyfikacja jego wag. Przy $\lambda = 1$ tylko zwycięski neuron dokonuje aktualizacji wag, co upodabnia algorytm NG do WTA [4].

Algorytm gazu neuronowego, według niektórych prac naukowych [15] jest skuteczniejszy niż algorytmy WTM i WTA, zwłaszcza w przypadku nierównomiernie rozmieszczonych wektorów wejściowych. Implementacja metody może jednak wiązać się z większymi kosztami i wymagać wyższej mocy obliczeniowej, między innymi ze względu na konieczność sortowania neuronów

3.5. Miary organizacji sieci

W ocenie jakości organizacji sieci samoorganizujących się stosuje się różne miary, opisane szczegółowo w literaturze [4, 8]. Na potrzeby tej pracy zostaną użyte dwie podstawowe miary: błąd kwantyzacji (ang. *quantization error*) oraz błąd topograficzny (ang. *topographic error*).

Błąd kwantyzacji

Błąd kwantyzacji służy do określania jakości odwzorowania danego wzorca wejściowego przez wagi neuronu uporządkowane w sieci. W procesie uczenia sieci dąży się do zmniejszania wartości błędu kwantyzacji.

Błąd kwantyzacji dla pojedynczego wektora wejściowego X_i mierzy odległość euklidesową pomiędzy X_i a najbliższym wektorem wag W_b neuronu zwycięskiego, według zadanej formuły [16]:

$$Q(X_i) = \|X_i - W_b(X_i)\| \tag{3.15}$$

Im mniejsza wartość $Q(X_i)$, tym lepiej neuron zwycięski dopasowuje się do danego wektora wejściowego. Błąd kwantyzacji ilustruje dopasowanie sieci neuronowej do konkretnego wzorca wejściowego. Patrząc na wartość pojedynczego błędu kwantyzacji nie można wyciągnąć wniosków na temat jakości wytrenowania całej sieci. Bardziej efektywną metodą oceny procesu wytrenowania jest średni błąd kwantyzacji, który mówi o tym jak dobrze sieć reprezentuje cały zbiór danych wejściowy.

Średni błąd kwantyzacji można opisać zależnością [4]:

$$\text{Q_error} = \frac{1}{N}\sum_{i=1}^{N} Q(X_i) = \frac{1}{N}\sum_{i=1}^{N} \|X_i - W_b(X_i)\|, \tag{3.16}$$

gdzie N jest liczbą wektorów wejściowych. Niska wartość średniego błędu kwantyzacji świadczy o dobrej jakości dopasowania neuronów sieci do rozkładu danych wejściowych.

Można również wyznaczyć maksymalny błąd kwantyzacji dla danego zbioru danych, opisany równaniem 3.17. Można go traktować jako wartość referencyjną, z która można porównywać otrzymane wyniki. W wyrażeniu tym n oznacza liczbę danych w zbiorze.

$$Q_{\text{max}} = \sqrt{n}. \tag{3.17}$$

Błąd topograficzny

Błąd topograficzny mierzy stopień naruszenia spójności topologicznej w mapie WTM, czyli to, czy neurony najbliższe danemu wektorowi wejściowemu X również są jego sąsiadami na mapie. Jest on definiowany jako odsetek wektorów wejściowych, dla których drugi najbliższy neuron nie jest sąsiadem neuronu zwycięskiego:

$$\text{T}_{\text{error}} = \frac{1}{N}\sum_{i=1}^{N} T(X_i), \tag{3.18}$$

gdzie $T(X_i) = 1$ jeśli drugi najbliższy neuron dla X_i nie jest sąsiadem neuronu zwycięskiego, a $T(X_i) = 0$ w przeciwnym razie.

Wysoki błąd topograficzny wskazuje na naruszenie topologii sieci, co oznacza słabe odwzorowanie relacji sąsiedztwa między danymi.

3.6. Przykłady możliwości zastosowania sieci SOM

Samoorganizujące się sztuczne sieci neuronowe są uniwersalnym narzędziem, znajdującym zastosowanie w wielu dziedzinach nauki i przemysłu. Skala ich zastosowań w znacznej mierze zależy od kreatywności oraz wyobraźni projektanta. W pracy [7] wymieniono kilka podstawowych zastosowania sieci SOM, w tym klasteryzację (ang. *clustering*) i kwantowanie wektorowe (ang. *vector quantization*).

Jednym z najczęstszych zastosowań sieci samoorganizujących się jest rozwiązywanie problemu klasteryzacji danych. Proces ten polega na grupowaniu danych wejściowych w klasy, ze względu na ich wzajemne podobieństwo, bez wcześniejszej informacji o ich istnieniu lub liczbie [7]. Można to rozumieć jako ekstrakcje kluczowych cech z wielowymiarowych danych i odnajdywanie elementów podobnych w zbiorze danych. W przypadku sieci sieci SOM, po wykonaniu treningu, neurony dopasowane są do danych uczących w taki sposób, że wagi pojedynczego elementu sieci można traktować jako reprezentantów różnych klas. Faza właściwej pracy sieci polega na odnajdywaniu najlepiej dopasowanego neuronu (ang. *Best Matching Unit*, skrót BMU) [8]. Znalezienie go polega na wyborze takiego neuronu, który jest najbardziej podobny do wzorca wejściowego, z użyciem wybranej miary podobieństwa. Po znalezieniu można założyć, że dana wejściowa należy do klasy reprezentowanej przez ten neuron.
Jedną z istotnych zalet stosowania sieci SOM w zadaniach klasteryzacji jest ich zdolność do zachowania topologii danych wejściowych [5]. Jest to kluczowa cecha, ponieważ można za jej pomocą ocenić stopień podobieństwa danych klas między sobą, tylko za pomocą analizy położeń BMU reprezentujących konkretne klasy. Ta właściwość jest szczególnie użyteczna z uwagi na prostotę i intuicyjność interpretacji zależności między klasami, nawet dla użytkownika nierozumiejącego zasad działania algorytmu [9].
Za pomocą klasteryzacji danych można również pośrednio rozwiązać inny problem - identyfikacje wartości odstających od klas, czyli tak zwanych outlierów. Można nazwać to również detekcją anomalii [3]. Realizować to można poprzez analizę podobieństwa między wektorem danych, a jego BMU. W przypadku stwierdzenia, że odległość jest za duża, czy to poprzez średnią odległość innych danych z tym samym BMU, czy to poprzez zastosowania odległości progowej, można stwierdzić, że dana nie reprezentuje żadnej z klas. To zastosowanie bywa niezwykle istotne, gdyż pozwala na wykrywanie nietypowych danych. Ma to kluczowe znaczenie między innymi w takich dziedzinach jak finanse, medycyna czy bezpieczeństwo systemów [8].
Niniejsza praca opiera się na zastosowaniu sieci SOM w zadaniach klasyfikacji danych. Klasyfikacje można rozumieć jako rozszerzenie zadania klasteryzacji, ponieważ klasyfikacja polega na dopasowaniu wzorca wejściowego do jednej z wcześniej zdefiniowanych klas.
W literaturze opisano wiele implementacji sieci Kohonena w zadaniach klasteryzacji, wykorzystywanych w przeróżnych i niezwiązanych wzajemnie dyscyplinach, takich jak finanse, biologia, przetwarzanie obrazów, czy analiza obciążeń sieci energetycznych. Zastosowanie te zostały opisane między innymi w pozycjach [9, 7, 5].

Kwantowanie wektorowe polega na aproksymacji pewnego zbioru danych za pomocą dużo mniejszej liczby reprezentantów, stanowiących uogólnienie grup występujących w danych. W przypadku użycia sieci SOM zamiast terminu kwantyzacji wektorowej używa się czasem sformułowania angielskiego *Learning Vector Quantization*, skrót LVQ [8], w celu podkreślenia obecności fazy uczenia w tym procesie. Po zakończeniu treningu neurony stają się ogólnymi reprezentantami poszczególnych klas danych. Wagi neuronów pełnią rolę wartości reprezentatywnych, opisujące

szersze grupy wektorów wejściowych, dla których konkretny neuron stanowi najlepsze dopasowanie (czyli BMU). [1] Kwantowanie wektorowe użyteczne jest zwłaszcza w zadaniach redukcji wymiarowości danych, przykładowo w zadaniach kompresji sygnałów i obrazów. Zastosowanie to zaprezentowano między innymi w [3].

Należy dodatkowo nadmienić, że sieci SOM mogą znaleźć zastosowanie jako narzędzia przy konstrukcji bardziej zaawansowanych modeli uczenia maszynowego [3]. Istnieje możliwość ich wykorzystania jako narzędzie do analizy danych, gdzie dokonują redukcji wymiarowości lub wstępnej segmentacji, co upraszcza późniejsze przetwarzanie inną techniką.
W literaturze można znaleźć wiele takich przykładów, między innymi w wykorzystaniu jako narzędzie wspierające diagnostykę medyczną. Przykładów w tej dziedzinie można znaleźć wiele, w tym takie, które łączą sieci SOM z algorytmem Fuzzy K-Mean, zaproponowano w pracy [17].
Koncepcja sieci samoorganizujących się jest wciąż rozwijana, co prowadzi do jej zastosowania w bardziej złożonych modelach. Przykładem są sieci Deep SOM (skrót DSOM), czyli modele stanowiące połączenie sieci SOM z innymi warstwami sieci neuronowych, takimi jak między innymi sieci splotowe (konwolucyjne). Rozwiązanie zawierające rozszerzenie tego konceptu, o algorytm nazwany przez autorów E-DSOM (ang. *Extended Deep SOM*, rozszerzony deep som), zaprezentowano w pracy [18].

3.7. Podsumowanie rozdziału

Prezentowany rozdział, wraz z rozdziałem 2, stanowi niezbędne przygotowanie teoretyczne do pełnego zrozumienia niniejszej pracy. Opisane zostały takie zagadnienia, jak różne topologie sieci, funkcje sąsiedztwa czy różne miary podobieństwa wektorów danych wejściowych. Zagadnienia te stanowią fundament teoretyczny implementacji programowej, opisanej w kolejnym, tj. 4 rozdziale niniejszej pracy.

4. PROPONOWANA IMPLEMENTACJA SIECI SOM

Podstawowym celem niniejszej pracy jest analiza możliwości wykorzystania sieci Kohonena w zastosowaniach związanych z klasyfikacją danych. W celu umożliwienia wykonania badań zaprojektowano i zaimplementowano od nowa sieć WTM. Decyzja wykonania własnego rozwiązania, pomimo istnienia gotowych pakietów i bibliotek, wynikała z potrzeby całkowitej kontroli nad strukturą algorytmu uczenia sieci. Dodatkowym atutem zaproponowanego rozwiązania był większy wybór dostępnych parametrów, takich jak różne funkcje sąsiedztwa, metody zmiany promienia sąsiedztwa, czy funkcje aktualizacji współczynnika uczenia. Zwiększona pula dostępnych parametrów umożliwiła badania sieci w konkretnych zastosowaniach, a także, co jest również ważne w szerszym spektrum.

Na początku opisu implementacji sieci WTM zostaną przedstawione ogólnodostępne rozwiązania sieci Kohonena. Następnie, opisane zostaną użyte narzędzia programistyczne oraz biblioteki wykorzystane w implementacji programowej sieci neuronowej. W dalszej części zostanie opisane wykonane rozwiązanie, wraz z dokładną strukturą algorytmu uczenia. Następnie zaprezentowane zostaną kluczowe parametry, wpływające bezpośrednio na proces uczenia sieci SOM wraz z zasadami ich doboru.

4.1. Dostępne implementacje programowe sieci SOM

W literaturze oraz w środowiskach programistycznych występuje kilka bibliotek oraz pakietów pozwalających na implementacje sieci SOM oraz ich trenowanie. W tym kontekście pojęcie biblioteka lub pakiet rozumieć można jako narzędzia rozszerzające podstawowe zastosowania języka programowania o gotowe moduły i funkcje umożliwiające proste tworzenie sieci SOM, bez konieczności implementacji ich od podstaw. Wykorzystanie takich narzędzi pozwala na użycie sieci SOM bez dogłębnej znajomości ich teorii i matematycznych podstaw, skupiając się jedynie na ich praktycznych zastosowaniach, takich jak analiza danych, wizualizacja struktur, czy segmentacja zbiorów danych.

Jednym z takich pakietów jest pakiet Minisom [19], napisany w języku Python, dostępny na licencji MIT. Pakiet ten jest minimalistyczną implementacją sieci samoorganizujących się, opartą na podstawowej bibliotece python'a - Numpy. Wykorzystanie pakietu Numpy pozwala na wysoce zoptymalizowane obliczenia macierzowe i wektorowe oraz inne operacje z zakresu algebry liniowej. Numpy wykorzystany jest ze względu na wyjątkową wydajności, ponieważ większość jego funkcji jest zaimplementowana w językach niskopoziomowych, takich jak C i Fortran. Powoduje to, że obliczenia wykonywane z użyciem Numpy są znacznie szybsze niż analogiczne funkcjonalności zaimplementowane w oparciu o standardowe funkcje Pythona.
Minisom oferuje prostotę użytkowania, umożliwiając proste tworzenie sieci samoorganizujących, także dla użytkownika o niższym poziomie znajomości tematu. Istnieje możliwość dostosowania wielu parametrów, takich jak niektóre funkcje sąsiedztwa, metody zmiany promienia obszaru sąsiedztwa, wymiary siatki SOM czy topologie sąsiedztwa. Pakiet ten nie oferuje jednak niektórych kluczowych funkcjonalności, takich jak możliwość wykonania równoległych obliczeń za pomocą procesorów graficznych (GPU) czy wsparcie dla pracy z bardzo dużymi zbiorami danych, co ogranicza jego zastosowanie w bardziej wymagających projektach obliczeniowych. Dodatkowo nie zapewnia wystarczająco szerokiego wyboru parametrów, w szczególności funkcji sąsiedztwa, co

uniemożliwia jego wykorzystanie w niniejszej pracy.

Istnieją również biblioteki pozwalające na tworzenie i trening sieci SOM w takich języków jak MATLAB i R. W środowisku MATLAB sieci samoorganizujące się nie są zaimplementowane bezpośrednio jako osobny pakiet, lecz istnieje możliwość tworzenie ich za pomocą Deep Learning Toolbox. Szersza funkcjonalność tego pakietu została opisana w pozycji [20]. Deep Learning Toolbox umożliwia proste definiowanie struktury sieci oraz jej trening. Użytkownik ma możliwość skorzystania z graficznego interfejsu użytkownika przy wyborze architektury sieci. Istnieje również możliwość wykorzystania procesorów graficznych do wykonywania obliczeń, co może przyśpieszyć proces trenowania sieci. Pakiet ten udostępnia również wiele narzędzi do wizualizacji wyników zarówno w procesie treningu sieci, jak i w czasie etapu właściwej pracy. W języku R istnieje pakiet Kohonen dedykowany do tworzenia i treningu sieci SOM. Oferuje możliwość definiowania architektury sieci, włączając w to wybór topologii i różnych funkcji sąsiedztwa oraz rozbudowane narzędzia do wizualizacji wyników. Pakiet ten nie wspiera możliwości wykonywania równoległych obliczeń za pomocą procesorów graficznych.
Zarówno pakiet Deep Learning Toolbox do języka MATLAB jak i Kohonen do R oferują wiele przydatnych funkcjonalności do tworzenia sieci Kohonena. Wykazują jednak pewne ograniczenia, takie jak wąski wybór funkcji sąsiedztwa, czy też funkcji zmiany współczynnika uczenia i promienia sąsiedztwa. Dodatkowo nie pozwalają na modyfikacje algorytmów uczenia, co uniemożliwia (lub mocno ogranicza) wykorzystanie ich w niniejszej pracy.

4.2. Wykorzystane narzędzia programistyczne

Zaproponowana implementacja programowa sieci samoorganizujących się została wykonana w języku Python. Wybór tego konkretnego języka wynikał z jego wszechstronności i wysokiej popularności w wykorzystaniu przy tworzeniu rozwiązań z dziedziny uczenia maszynowego. Język Python jest językiem programowania wysokiego poziomu z prostą i czytelną składnią. Z punktu widzenia programisty posiada wiele przydatnych funkcjonalności, takich jak dynamiczny system typów zmiennych, automatyczne zarządzanie pamięcią czy szeroki wybór dostępnych pakietów, poszerzających podstawowe wykorzystanie języka. Wspiera on również wiele paradygmatów programowania, takich jak programowanie obiektowe, szeroko wykorzystane w zaproponowanej implementacji.

W zaproponowanej rozwiązaniu wykorzystano kilka pakietów, rozszerzających podstawowe funkcjonalności programu. Są to:

- Pytorch,
- Pandas,
- Matplotlib.

PyTorch jest biblioteką dedykowaną do uczenia maszynowego rozwijaną głównie przez firmę Meta, począwszy od roku 2016, aż do dnia dzisiejszego. Jej głównym zastosowaniem jest kompleksowe tworzenie, trenowanie i testowanie zarówno prostych, jak i bardziej rozbudowanych, sieci neuronowych [21]. W przypadku tego pakietu stosuje się raczej angielskie określenie *framework* (tłumaczenie: rama), niż termin biblioteka. Inne nazewnictwo służy podkreśleniu zastosowaniu Pytorch-a jako podstawy konstrukcji zaawansowanych systemów uczenia maszynowego. Pakiet wyróżnia się intuicyjną składnią, bardzo zbliżoną do stosowanej w języku Python, co znacząco ułatwia tworzenie modeli. PyTorch pozwala na efektywne wykorzystanie mocy obliczenio-

wej zarówno procesorów graficznych (GPU), jak i jednostek centralnych (CPU), w zależności od ich dostępności. Możliwość ta może znacząco przyspieszyć trenowanie sieci neuronowych, szczególnie w przypadku rozbudowanych zbiorów danych [21].

W bibliotece dostępne są gotowe implementacje różnych typów sieci neuronowych, takich jak sieci jednokierunkowe, liniowe czy rekurencyjne. Użytkownik ma możliwość dostosowania szerokiego zakresu parametrów, w tym funkcji aktywacji oraz liczby neuronów w warstwach. Dodatkowo PyTorch umożliwia łączenie różnych rodzajów warstw w obrębie jednego modelu [21]. W kontekście niniejszej pracy należy wspomnieć, że sieci SOM nie mają gotowej implementacji zawartej w bibliotece Pytorch. Do ich stworzenia wykorzystano podstawowe narzędzie Pytorch-a - tensor. Tensor jest wielowymiarową strukturą danych, którą można traktować jako wielowymiarową macierz. Umożliwia ona przechowywanie danych i wykonywanie na nich operacji matematycznych różnego rodzaju. Przewagą stosowania tensora nad wielowymiarowymi macierzami jest możliwość wykonywania operacji zarówno na CPU, jak i GPU. W zaproponowanej implementacji tensor posłużył do reprezentacji wag sieci Kohonena oraz do przekazywania zbioru wzorców wejściowych na wejście sieci neuronowej, w celu ich przetworzenia.

Pandas jest biblioteką ułatwiającą wczytywanie danych oraz ich wstępne przetwarzanie, w celu przekazania ich w dogodnej formie bezpośrednio do algorytmu uczącego. Biblioteka rozwijana jest od 2008 roku. Dzięki prostemu interfejsowi i wsparciu dla różnorodnych formatów danych, stanowi ona jedno z najbardziej popularnych narzędzi, niezbędnym w procesie trenowania modeli uczenia maszynowego. Kluczowym elementem biblioteki jest struktura danych nosząca nazwę DataFrame, która umożliwia przetwarzanie danych w formie tabeli. W oparciu o tę strukturę użytkownik ma możliwość wykonywania różnorodnych operacji, takich jak filtrowanie, grupowanie czy uzupełnianie brakujących wartości.
W proponowanej rozwiązaniu wykorzystano DataFrame do wczytania danych i podziału ich na zbiór uczący i testowy.

Matplotlib jest narzędziem służącym do wizualizacji danych w formie graficznej. Pakiet ten umożliwia generowanie wykresów jedno-, dwu- lub trójwymiarowych, zarówno w formie statycznej jak i animowanej. Umożliwia wykreślanie klasycznych charakterystyk funkcyjnych oraz takich reprezentacji jak histogramy czy wykresy słupkowe. Obsługuje różne formaty danych, łącznie z DataFrame z Pandas, co pozwala na efektywne i czytelne przedstawianie wyników w formie graficznej. W zaimplementowanej sieci WTM użyto go do wizualizacji wyników, głównie do wykreślania map zwycięzców.

4.3. Opis implementacji programowej

Głównym elementem w zaimplementowanej sieci WTM jest klasa ”SelfOrganizingMap”. Zawiera ona podstawową logikę inicjalizacji i trenowania sieci. Po zakończonej fazie treningu obsługuje ona również fazę właściwej pracy sieci. Oprócz tej klasy stworzono również klasę ”VisualizeSOMResults”, która pozwala prezentować wyniki wytrenowanej sieci.

Inicjalizacja sieci samoorganizującej się

Przy realizacji obiektu klasy ”SelfOrganizingMap” wykonywany jest konstruktor klasy. Użytkownik ma możliwość wyboru szeregu parametrów, zaprezentowanych poniżej:

1. **Funkcje sąsiedztwa:**
    - Funkcja Gaussa, dana wzorem 3.7

- Funkcja progowa, dana wzorem 3.6
- Funkcja trójkątna, dana wzorem 3.9
- Funkcja eksponencjalna, dana wzorem 3.8

2. **Topologie sieci, opisane w 3.2.2:**
   - Prostokątna z 8 sąsiadami
   - Heksagonalna
3. **Miary odległości:**
   - Odległość euklidesowa, dana wzorem 3.1
   - Odległość Manhattan, dana wzorem 3.2
   - Iloczyn skalarny, dany wzorem 3.3

Program wyposażony jest w mechanizm mający za zadanie walidację wprowadzonych przez użytkownika parametrów. W przypadku wybrania nieobsługiwanego parametru program wyświetli komunikat błędu oraz zakończy swoje działanie.
Po wykonaniu konstruktora obiektu program przeprowadza inicjalizację sieci SOM na podstawie wybranych parametrów, wraz z wektorami wag neuronów. Wektorom wag na tym etapie przypisuje się wartości pseudolosowe, z rozkładu od zera do jeden.

Trening sieci SOM z wykorzystaniem algorytmu WTM

Po skutecznym wykonaniu inicjalizacji należy wytrenować sieć neuronową. W tym celu zaimplementowano metodę o nazwie ”train”, która przeprowadza trening sieci SOM za pomocą klasycznej wersji algorytmu zwycięzca bierze większość. Metoda ta, jako argument wejściowy przyjmuje zbiór danych uczących, Inne niezbędne informację odnośnie procesu treningu inicjalizowane są podczas tworzenia obiektu i zapisane są w jego parametrach. Metoda ”train” zwraca wartości błędów kwantyzacji obliczanych po każdej epoce treningu. W czasie jej trwania modyfikują się również wagi poszczególnych neuronów, zapisane w tensorze jako parametr obiektu. Algorytm uczenia został szczegółowo przedstawiony za pomocą pseudokodu na rysunku 4.1.

Algorithm Uczenie sieci SOM za pomocą algorytmu WTM Przebieg algorytmu WTM w uczeniu sieci SOM

Dane wejściowe: Zbiór treningowy obrazów (X)

Dane wyjściowe: Lista średnich błędów kwantyzacji Q_E

1: $Q_E \leftarrow []$ ▷ Inicjalizacja tablicy błędów kwantyzacji
2: **for** każda epoka e **do**
3: **for** każdy wektor wejściowy $x \in X$ **do**
4: $D(x, W_j) = \|x - W_j\|$ ▷ Odległości między x a wszystkimi wagami neuronów
5: $BMU = \arg\min_j D(x, W_j)$ ▷ Wyznaczanie zwycięzcy (BMU)
6: $\alpha = G(R, d(i, j))$ ▷ Obliczanie wartości α za pomocą funkcji sąsiedztwa
7: $\Delta W_j = \eta \cdot \alpha \cdot (x - W_j)$ ▷ Obliczanie zmiany wag W_j
8: $W_{j+1} \leftarrow W_j + \Delta W_j$ ▷ Aktualizacja wag neuronów w sieci po każdej iteracji
9: **end for**
10: $Q_E(e) = \frac{1}{len(X)} \sum_{x \in X} \min_j \|x - W_j\|$ ▷ Obliczanie średniego błędu kwantyzacji dla epoki e
11: Dodaj $Q_E(e)$ do Q_E ▷ Rozszerzenie tablicy błędów o $Q_E(e)$
12: Zmniejszanie wartości promienia sąsiedztwa oraz współczynnika uczenia
13: **end for**
return Q_E ▷ Zwrócenie listy błędów kwantyzacji

Rysunek 4.1: Opis przebiegu algorytmu uczenia SOM

W zaproponowanym algorytmie (rys. 4.1) wyróżnić można kilka podstawowych elementów. Są nimi obliczenie odległości między wektorami wag neuronów a wzorcem wejściowym, znalezienie neuronu zwycięzcy i modyfikacja wag neuronów z uwzględnieniem funkcji sąsiedztwa i współczynnika uczenia.

Przy okazji algorytmu uczenia dodano możliwość wyboru sposobu zmiany promienia sąsiedztwa. W literaturze spotyka się różne interpretacje, w kontekście tego, czy promień sąsiedztwa jest tylko liczbą naturalną [8], czy może być również dodatnią liczbą zmiennoprzecinkową [5]. W niniejszej pracy zdecydowano się na sprawdzenie możliwości obydwu podejść, dlatego też dostosowano algorytm uczenia do obsługi obu możliwości.

W zaproponowanej implementacji podczas procesu właściwej pracy sieci wykorzystuje się metodę o nazwie `"get_bmu"`. Metoda ta przyjmuje zbiór wzorców wejściowych i dla każdego z nich zwraca pozycję neuronu zwycięzcy w mapie SOM. Wynik ten może zostać następnie przekazany do metod z klasy ”VisualizeSOMResults”, w celu reprezentacji graficznej wyniku.

Zaproponowano również funkcję do obliczania średnich błędów kwantyzacji oraz średniego błędu topograficznego, w celu ewaluacji modelu zarówno w czasie procesu treningu (obserwacja szybkości zmian błędu może świadczyć o doborze parametrów), jak i podczas właściwej pracy.

Narzędzia do wizualizacji rezultatów

Dla graficznej prezentacji wyników stworzono klasę ”VisualizeSOMResult”. Klasa ta zawiera i dwie metody klasy. Pierwszą z nich jest metoda wykreślająca charakterystykę wartości błędu kwantyzacji od epoki uczącej. Charakterystyka ta wykorzystywana jest po procesie uczenia i pomaga w

ocenie poprawności doboru architektury oraz parametrów sieci SOM.

Druga z metod została opracowana w celu wizualnej oceny jakości wytrenowania sieci SOM. Polega ona na tworzeniu i analizie tzw. mapy zwycięzców (ang. *WinMap*) [19], która jest popularnym narzędziem stosowanym w praktycznych implementacjach sieci samoorganizujących się.

Mapa zwycięzców to graficzna reprezentacja dwuwymiarowej siatki SOM, na której przedstawione są etykiety wzorców wejściowych. Etykiety są wyświetlane w pozycjach odpowiadających współrzędnym ich neuronów zwycięzców na mapie, z przesunięciem o 0.5 zarówno wzdłuż osi X jak i Y, w celu poprawy wizualizacji (ma to na celu zwiększenie czytelności) [20]. Tego typu wizualizacja umożliwia ocenę jakości klasteryzacji poprzez obserwację rozmieszczenia etykiet na mapie.

Oczekiwanym rezultatem jest wizualne grupowanie etykiet w spójne obszary mapy SOM. Bliskość etykiet na mapie sugeruje, że odpowiadające im wzorce wejściowe są do siebie podobne, co świadczy o poprawnym działaniu sieci. Przykład takiego grupowania przedstawiono w sekcji 5.2 na rysunku 5.1, gdzie wykorzystano model sieci SOM o rozmiarach 6 na 6 neuronów, z topologią heksagonalną i funkcją sąsiedztwa Gaussa. Model trenowany był na zbiorze Irys, który został szerzej opisany w 5.2. Ewaluacja została przeprowadzona na danych testowych.

4.4. Opis parametrów sieci SOM wraz z zasadami doboru

Na jakość wytrenowania sieci samoorganizującej ma wpływ szereg parametrów. Należą do nich między innymi: wymiary sieci, zastosowana topologia sąsiedztwa, użyta funckja sąsiedztwa, początkowy promień sąsiedztwa czy początkowa wartość współczynnika uczenia. Ostatnie dwa z wymienionych parametrów zmniejszają się w trakcie procesu uczenia, dlatego istotny jest również dobór funkcji regulującej ich stopniowe zmniejszanie.

Metody doboru początkowej wartości parametrów wynikają raczej z doświadczenia projektanta sieci, niż ze ściśle określonych reguł zapisanych w literaturze. Metody opisane poniżej wynikają z połączenia informacji zawartych w literaturze, w takich pozycjach jak [5, 19] oraz praktycznych doświadczeń nabytych podczas badań.

W kontekście wymiarów sieci często stosuje się zasadę, według której liczba neuronów w sieci powinna, w przybliżeniu, być równa $5 \cdot k,\ gdzie\ k = \sqrt{\text{liczba danych w zbiorze}}$ [18]. Reguła ta została zastosowana również w niniejszej pracy, przy czym wykorzystano siatki o jednakowej liczbie neuronów zarówno w pionie, jak i poziomie.

Zgodnie z danymi z literatury (np. [8]) początkowy promień sąsiedztwa powinien obejmować większą część siatki, zmniejszając się w czasie, w celu umożliwienia lokalnego dostrajania wag w końcowym etapie treningu. Dane te zostały jednak podważone w pracy [4], gdzie udowodniono, że początkowy promień sąsiedztwa może być zdecydowanie mniejszy niż połowa mapy, nie pogarszając, a też często poprawiając końcowe wyniki procesu uczenia. Na potrzeby jednak niniejszej pracy założono, że początkowy promień sąsiedztwa równy jest połowie największego wymiaru założonej siatki (oznacza to, że dla siatki 30 x 30 początkowy promień równy jest 15). Promień zmniejsza się tak, aby na koniec procesu treningu osiągnąć wartość zero, lub w niektórych podejściach do jeden. Przy badaniach używających całkowitych wartości promienia sąsiedztwa, wartości zaokrągla się do najbliższej liczby naturalnej.

Współczynnik uczenia jest parametrem określającym dynamikę zmian w danej epoce treningowej. Na początku procesu treningu wskazanym jest, aby wartość współczynnika była relatywnie wysoka, rzędu od 0,2 do 0,5. Podobnie, jak w przypadku promienia sąsiedztwa, wartość współczynnika uczenia powinna maleć, aż do wartości bliskiej zeru. Dobór parametru uczenia

przeprowadzany był w sposób doświadczalny, poprzez analizę osiągniętych wyników po procesie treningu, oraz poprzez analizę zmiany błędu kwantyzacji podczas kolejnych epok procesu uczenia.

Zarówno wartość promienia sąsiedztwa, jak i współczynnika uczenia zmienia się za pomocą zależności funkcyjnej. Są to funkcje zależne od epoki treningowej. Funkcje te posiadają również stały współczynnik, którego wartość zależna jest od promienia sąsiedztwa.
W proponowanej implementacji użyto 4 funkcji zmiany promienia sąsiedztwa i współczynnika uczenia:

1. Funkcja wymierna, zaprezentowana zależnością (4.1):

$$h_(k) = \frac{C \cdot \mathsf{A}}{C + \mathsf{k}} \tag{4.1}$$

2. Funkcja dążąca do jedności, zaprezentowana zależnością (4.2):

$$h_(k) = \frac{\mathsf{A}}{1 + \mathsf{k} \cdot \frac{\mathsf{A}-1}{\mathsf{T}}} \tag{4.2}$$

3. Funkcja liniowa, zaprezentowana zależnością (4.3):

$$h_(k) = \mathsf{A} \cdot \left(1 - \frac{\mathsf{k}}{\mathsf{T}}\right) \tag{4.3}$$

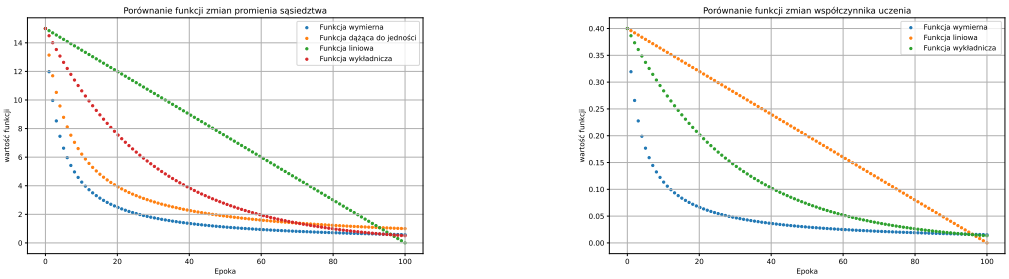
4. Funkcja wykładnicza, zaprezentowana zależnością (4.4):

$$h_(k) = \mathsf{A} \cdot e^{-\frac{\mathsf{k}}{\mathsf{C}}} \tag{4.4}$$

W zaprezentowanych powyżej zależnościach A oznacza odpowiednio wartość początkową współczynnika uczenia, bądź początkowy promień, T oznacza całkowitą liczbę epok treningowych. Epoka jest to argument funkcji, a C jest pewną stałą zależną od liczby danych w zbiorze uczącym. Należy również podkreślić, że nazwy dla funkcji zostały nadane wyłącznie na potrzeby niniejszej pracy i stanowią odpowiedniki angielskich nazw, które są powszechnie używane w literaturze dotyczącej praktycznych zastosowań sieci samoorganizujących się [5].

Implementacja czterech funkcji zmiany promienia miała na celu umożliwienie przeprowadzenia badań i sprawdzenie, która z nich umożliwi osiągnięcie jak najlepszych rezultatów. W literaturze najczęściej stosuje się funkcję dążącą do jedności oraz wymierną dla zmiany promienia sąsiedztwa, natomiast dla współczynnika uczenia częściej stosuje się funkcję wykładniczą. Dla współczynnika uczenia nie stosuje się funkcji dążącej do jedności, z uwagi na jego początkową wartość, która jest mniejsza od jedności. Przy niezłożonych zbiorach danych używa się również często funkcji liniowej do zmiany tych parametrów.

Na rysunku 4.2 zaprezentowano porównanie wykresów wszystkich wymienionych w 4.4 funkcji. Porównanie dla promienia sąsiedztwa przedstawiono na 4.2a. Założono początkową wartość promienia równą 15 oraz 100 epok treningowych. Rysunek 4.2b przedstawia porównanie dla współczynnika uczenia, założono początkową wartość równą 0,4 oraz 100 epok treningowych.



(a) Funkcje zmian promienia sąsiedztwa

(b) Funkcje zmian współczynnika uczenia

Rysunek 4.2: Wykresy zmian promienia sąsiedztwa oraz współczynnika uczenia

4.5. Podsumowanie rozdziału

Prezentowany rozdział miał na celu wyjaśnienie sposobu implementacji samoorganizującej się sieci neuronowej typu WTM. Dokonano przeglądu ogólnodostępnych bibliotek i pakietów dedykowanych różnym językom programowania, implementujących sieci Kohonena. Ponadto przedstawiono wykorzystane narzędzia, wraz z uzasadnieniem ich użycia, omówiono szczegóły implementacji oraz metodologię doboru parametrów wpływających na działanie sieci SOM.
W kolejnym rozdziale pracy zaprezentowane zostaną wyniki przeprowadzonych badań, wraz z metodyką ich prowadzenia.

5. PRZEPROWADZONE BADANIA

Głównym celem niniejszej pracy były badania wpływu wybranych funkcji sąsiedztwa na możliwość późniejszego wykorzystania sieci samoorganizujących w zadaniach klasyfikacji danych. Do przeprowadzenia badań wykorzystano implementację opisaną w rozdziale 4.
Badania przeprowadzono na trzech popularnych zbiorach uczących: ”MNIST”[22], ”IRIS”[23] i Cardiotocography[24]. Każdy z tych zbiorów danych zostanie szczegółowo opisany w osobnej sekcji prezentowanego rozdziału.

W badaniach skupiono się na analizie wpływu wybranej funkcji sąsiedztwa na wartość błędu kwantyzacji, błędu topograficznego i na ocenie jakości klasteryzacji danych na podstawie uzyskanych map zwycięzców. Porównano również dwie rozbieżne kwestie podnoszone w literaturze – dotyczące natury promienia w sieciach SOM. Rozważano, czy promienie sąsiedztwa powinny przyjmować wyłącznie wartości całkowite, tak jak opisano to w pozycji [8], czy też mogą być liczbami niecałkowitymi, tak jak przedstawiono to w [5]. Porównania dokonano dla dwóch topologii sieci - prostokątnej z ośmioma sąsiadami i heksagonalnej.

W celu realizacji postawionych założeń przyjęto jednolitą metodologię badań, opisaną w sekcji 5.1.

5.1. Metodologia badań

Procedura wykonywania badań była jednakowa dla każdego zbioru uczącego. Początkowym etapem było wczytanie danych do programu i wstępne ich przetworzenie. Na tym etapie zbiór danych dzielony był na zbiór uczący i zbiór testowy, w proporcjach osiemdziesiąt (80) do dwudziestu (20) procent. Wyjątkiem tutaj były badanie na zbiorze IRIS, co uzasadniono w sekcji 5.2. Następnie wszystkie dane były normalizowane, do zakresu od zera do jeden. Normalizacja przebiegała według wzoru 5.1.

$$z' = a_g + \frac{(z - z_{\text{min}})}{(z_{\text{max}} - z_{\text{min}})} \cdot (b_g - a_g), \tag{5.1}$$

gdzie z oznacza oryginalną wartość w zbiorze danych, z' to przeskalowana wartość, z_{min} i z_{max} reprezentują odpowiednio minimalną i maksymalną wartość w kolumnie danych, zaś a_g i b_g to granice przedziału skalowania ($a_g = 0$ oraz $b_g = 1$).

Liczba epok oraz parametry sieci były dostosowywane indywidualnie dla każdego zbioru uczącego, zgodnie z zasadami zaprezentowanymi w sekcji 4.4. Wartości tych parametrów pozostawały takie same dla wszystkich funkcji sąsiedztwa użytych przy treningu i ewaluacji modeli. Również funkcje zmiany współczynnika uczenia i zasięgu sąsiedztwa były jednolite w obrębie każdego zbioru uczącego. Pewną stałą wśród wszystkich badań była zastosowana miara odległości między wzorcami wejściowymi, a wektorami wag. Użyto miary euklidesowej, opisanej w sekcji 3.1, ze względu na jej popularność w innych pracach [8, 17, 13].

Ewaluacja modeli polegała na wyznaczeniu wartości średniego błędu kwantyzacji oraz błędu topograficznego, na danych testowych, dla każdego modelu, stworzonego z zastosowaniem innej funkcji sąsiedztwa w procesie uczenia. Tworzone były również mapy zwycięzców do wizualnej oceny jakości klasteryzacji.

W kolejnych sekcjach rozdziału zaprezentowano osiągnięte rezultaty. W każdej z sekcji 5.2, 5.3 oraz 5.4 opisano wykorzystany zbiór uczący wraz z uzasadnieniem wyboru. Umieszczono w nich także tabele z porównaniem wartości średnich błędów kwantyzacji i topograficznego oraz

mapę zwycięzców dla modelu z najmniejszym wynikami obu błędów. Zdecydowano się na prezentację tylko jednej z map zwycięzców, z uwagi na ilość wytrenowanych modeli.

5.2. Badania na zbiorze IRIS

Zbiór danych *Iris* jest jednym z najpopularniejszych zbiorów danych wykorzystywanych w zadaniach klasyfikacji i klasteryzacji. Przez swoją prostą strukturę, umożliwia wstępne zbadanie wszelkiego rodzaju algorytmów analizy danych, dlatego jest częstym przedmiotem badań [23]. Zbiór ten powstał w 1936 roku i zawiera 150 próbek opisujących 3 gatunki irysów (roślina ozdobna). Każdy wektor danych zawiera cztery cechy opisujące morfologię kwiatu tej rośliny: długość działki kielicha (*sepal length*), szerokość działki kielicha (*sepal width*), długość płatka (*petal length*) oraz szerokość płatka (*petal width*).
Dane wejściowe podzielone są na 3 klasy, reprezentujące różne gatunki rośliny:

- **Setosa**: 50 próbek, z etykietą ”0”,
- **Versicolor**: 50 próbek, z etykietą ”1”
- **Virginica**: 50 próbek, z etykietą ”2”.

Zbiór danych wydaje się z pozoru prosty do klasyfikacji, ze względu na swoją wielkość i ograniczoną liczbę cech. Problemy może jednak sprawiać specyficzny podział na klasy, występujący w tym zbiorze. Wynika to z faktu, że irysy z gatunku **Versicolor** mają podobną budowę morfologiczną, jak irysy z gatunku **Virginica**. Prowadzi to często do trudności w jednoznacznej klasyfikacji tych roślin.

Zbiór danych został wybrany z uwagi na niewielki rozmiar, równą liczność klas oraz ograniczoną liczbę cech. W badaniu skupiono się na ocenie działania sieci samoorganizującej się w takim zbiorze, a także na analizie jej zdolności do zachowania topologii danych. Zakłada się, że taka właściwość sieci SOM powinna prowadzić do wyraźnego oddzielenia danych należących do klasy *Setosa* od pozostałych, natomiast dane z klas *Versicolor* i *Virginica*, ze względu na ich podobieństwo, mogą częściowo się nakładać na mapie zwycięzców.

Do tworzenia modeli wykorzystano siatkę 8 na 8 neuronów, z początkowym promieniem równym 4 i początkową wartością współczynnika uczenia równą 0.3. Wykorzystano funkcję liniową, opisaną w sekcji 4.3, dla zmiany wartości promienia oraz funkcję wymierną, opisaną w sekcji 4.1, dla zmiany wartości współczynnika uczenia. Ze względu na ograniczoną liczbę danych w zbiorze, każda z sieci była trenowane przez 10000 epok.

W przypadku tego (dość małego zbioru) zbioru, dane nie były dzielone na zbiór uczący i testowy. Skutkowało to ewaluacją modeli na tych samych danych, na których przeprowadzone zostało uczenie sieci. Sieć Kohonena potraktowano tutaj jako narzędzie do analizy i klasyfikacji danych wyłącznie w obrębie zbioru uczącego. Takie zastosowanie sieci jest również popularne, między innymi w pozycji [17].

Tabela 5.1 przedstawia otrzymane wartości błędów kwantyzacji i topograficznych. Zastosowane skróty oznaczają kolejno: **hex**$_{ncp}$ – badania dla topologii heksagonalnej i niecałkowitych wartości promienia, **rect**$_{ncp}$ – badania dla topologii prostokątnej z ośmioma sąsiadami i niecałkowitych wartości promienia, **hex**$_{cp}$ – badania dla topologii heksagonalnej i całkowitych wartości promienia, oraz **rect**$_{cp}$ – badania dla topologii prostokątnej z ośmioma sąsiadami i całkowitych wartości promienia.

Tabela 5.1: Wartości błędów topograficznych i kwantyzacji z testów na zbiorze IRIS

funkcja sasiedztwa	Rodzaj błędu	hex$_{ncp}$	rect$_{ncp}$	hex$_{cp}$	rect$_{cp}$
Gaussa	błąd kwantyzacji	0.0837	0.0946	0.0931	0.0936
	błąd topograficzny	0.0063	0.1128	0.0338	0.0902
trójkątna	błąd kwantyzacji	0.0630	0.0626	0.0574	0.0576
	błąd topograficzny	0.0812	0.2066	0.1123	0.2130
prostokątna	błąd kwantyzacji	0.0942	0.0805	0.0858	0.0769
	błąd topograficzny	0.0096	0.1153	0.0420	0.1430
eksponencjalna	błąd kwantyzacji	0.0810	0.0769	0.0820	0.0741
	błąd topograficzny	0.0600	0.1133	0.0661	0.1532

5.2.1. Wnioski z badań na zbiorze IRIS

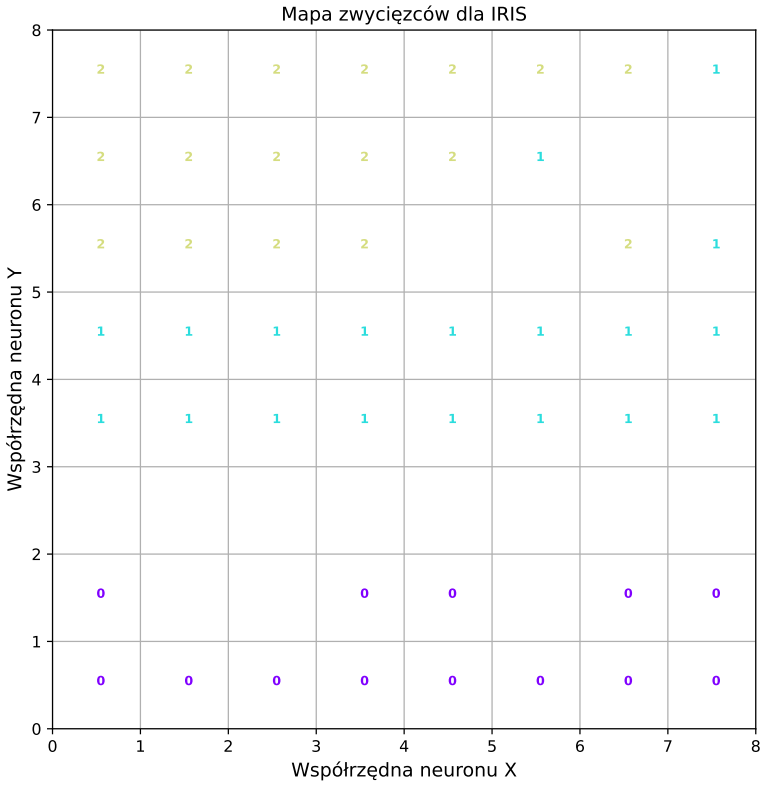
Maksymalny błąd kwantyzacji dla tego zbioru danych, zdefiniowany zależnością 3.17 przyjmuje wartość$\sqrt{4} = 2$. Najlepszą z uzyskanych wartości w przypadku tego błędu, wynosi 0.0574 i stanowi 2.87% wartości maksymalnej. Otrzymano ją dla zastosowania funkcji trójkątnej, z promieniem sąsiedztwa zmieniającym się w sposób całkowity oraz topologią heksagonalną. W tym wypadku wystąpił jednak znaczny błąd topograficzny, co przeniosło się bezpośrednio na rozkład klas na mapie zwycięzców. Podobnie, przy innych sieciach trenowanych za pomocą tej funkcji osiągnięto wysokie błędy topograficzne, które uniemożliwiają skuteczne stosowanie zaproponowanej funkcji trójkątnej w tym przypadku. Otrzymane rezultaty mogą wynikać z postaci funkcji trójkątnej stosowanej w zaproponowanej implementacji. Szersze skomentowanie wyników błędów otrzymanych przy trenowaniu sieci z użyciem funkcji trójkątnej przedstawiono w rozdziale 6.

Użycie funkcji eksponencjalnej pozwoliło na zmniejszenie wartości błędów topograficznych, w stosunku do ich wartości w przypadku stosowania funkcji trójkątnej. Zadowalające rezultaty osiągnięto także przy zastosowaniu funkcji progowej. W każdej możliwej kombinacji topologia - sposób zmiany promienia, osiągnięto rezultaty nieznacznie gorsze od funkcji Gaussa. Jest to dość nieoczekiwana obserwacja, która może sugerować, że dla prostych zbiorów danych stosowanie funkcji progowej jest warte rozważenia.

Najmniejsze wartości błędu kwantyzacji i topograficznego, wraz z najlepszymi rezultatami na mapie zwycięzców, osiągnięto przy stosowaniu funkcji Gaussa w topologii heksagonalnej wraz ze zmniejszającym się w sposób niecałkowity promieniem. Wynik takiego grupowania przedstawiono na rysunku 5.1. Przy zastosowaniu topologii prostokątnej również osiągano zadowalające wartości błędów, w zestawieniu z innymi funkcji sąsiedztwa. Zmiana sposobu określania promienia na całkowity spowodowała zwiększenie błędów topograficznych, tym samym pogarszając jakość grupowania na mapie zwycięzców.

Grupowanie klas zaprezentowane na rysunku 5.1 spełnia oczekiwania, wskazując na widoczne klastry odpowiadające poszczególnym etykietom danych. Szczególnie zauważalna jest strefa bez zwycięzców, między danymi z etykietą 0 a 1, co jest zgodne z teoretycznymi założeniami dotyczącymi zdolności sieci SOM do zachowywania relacji w danych. Takie rozmieszczenie odzwierciedla naturalne odseparowanie tych dwóch analizowanych gatunków irysów, wspomniane na wstępie tej sekcji. Z kolei dane z etykietami 1 i 2 reprezentowane są przez neurony położone blisko siebie, aż do przypadków nachodzenia na siebie. Prezentowana mapa zwycięzców wiernie

odwzorowuje rzeczywisty układ danych w zbiorze IRIS.
Osiągnięte rezultaty wskazują, że istnieje możliwość skutecznego wykorzystania sieci SOM do klasyfikacji danych o niewielkiej liczbie wymiarów i zrównoważonej liczności elementów w poszczególnych klasach, przy jednoczesnym zachowaniu ich zależności topologicznych, tak jak zostało to zaobserwowane przy analizie badań na zbiorze Iris.



Rysunek 5.1: Mapa zwycięzców otrzymana w wyniku testów na zbiorze IRIS

5.3. Badania na zbiorze Cardiotocography

Zbiór danych Cardiotocography zawiera dane medyczne pozyskane z badań kardiotokografi (CTG), analizujących akcję serca płodu ludzkiego. Informacje umieszczone w tym zbiorze zostały udostępnione przez Instytut Położnictwa i Ginekologii Uniwersytetu Medycznego w Porto w Portugalii [24]. Zbiór danych zawiera 2126 próbek, każda z 21 cechami, takimi jak liczba uderzeń serca na minutę, czy liczba ruchów płodu. Oprócz cech każda z wartości ma przypisaną jedną z trzech etykiet, przypisanych przez lekarzy położników:

- 1 - **Normalny (ang. Normal)**: oznaczający brak problemów zdrowotnych, liczność podzbioru danych wynosi 1655,
- 2 - **Podejrzany (ang. Suspect)**: wymagający dalszej obserwacji, liczność podzbioru danych wynosi 295,
- 3 - **Patologiczny (ang. Pathologic)**: wskazujący na możliwe komplikacje, liczność podzbioru danych wynosi 176.

Z punktu widzenia analizy danych zbiór uczący zawiera niewiele klas, za to dane posiadają wiele wymiarów. Należy również uwzględnić, że jedna z tych klas jest zdecydowanie dominująca. Wyboru dokonano zarówno ze względu na praktyczną możliwość wykorzystania sieci SOM jako narzędzia wspomagającego pracę w tak istotnej dziedzinie jak medycyna, jak i przez czysto techniczną potrzebę zbadania możliwości wykorzystania sieci przy analizie wielowymiarowych danych z niewielką liczbę klas, przy bardzo nierównomiernym rozkładzie liczby danych na klasę.

Do tworzenia modeli wykorzystano siatkę 14 na 14 neuronów, z początkowym promieniem równym 7, a początkową wartością współczynnika uczenia równą 0.4. Wykorzystano funkcję wykładniczą dla zmiany wartości promienia sąsiedztwa oraz funkcję wymierną dla zmiany wartości współczynnika uczenia. Ze względu na liczność zbioru danych uczących, każda z sieci była trenowana przez 1000 epok.

Wartości uzyskanych błędów kwantyzacji i błędów topograficznych przedstawione zostały w tabeli 5.2

Tabela 5.2: Wartości błędów topograficznych i kwantyzacji z testów na zbiorze Cardiotocography

funkcja sasiedztwa	**Rodzaj błędu**	**hex**$_{ncp}$	**rect**$_{ncp}$	**hex**$_{cp}$	**rect**$_{cp}$
Gaussa	błąd kwantyzacji	0.3482	0.3522	0.3460	0.3390
	błąd topograficzny	0.0281	0.0962	0.0398	0.1032
trójkątna	błąd kwantyzacji	0.3242	0.3222	0.3474	0.3141
	błąd topograficzny	0.2159	0.3556	0.2523	0.3638
prostokątna	błąd kwantyzacji	0.3661	0.3620	0.3564	0.3472
	błąd topograficzny	0.0680	0.1737	0.0986	0.1807
eksponencjalna	błąd kwantyzacji	0.3524	0.3496	0.3560	0.3470
	błąd topograficzny	0.0728	0.1995	0.0751	0.2078

5.3.1. Wnioski z badań na zbiorze Cardiotocography

Z uwagi na ilość wymiarów danych maksymalny błąd kwantyzacji, wyznaczony za pomocą zależności 3.17, przyjmuje wartość $\sqrt{21} \approx 4.58$.

Najmniejszy z otrzymanych błędów kwantyzacji, wynoszący 0.3141, osiągnięto przy zastosowaniu funkcji trójkątnej, wraz funkcją zmiany promienia przyjmującą wartości całkowite oraz prostokątna topologią sieci. Stanowi on 7.034% wartości maksymalnej. W przypadku tego badania błąd topograficzny ma jednak bardzo wysoką wartość, co skutkowało niepoprawnym grupowaniem na mapie zwycięzców.

Zastosowanie funkcji trójkątnej, w ogólności, przekładało się na niskie wartości błędów kwantyzacji, przy równoczesnym wzroście wartości błędów topograficznych, w stosunku do innych zastosowanych metod. Było to szczególnie widoczne przy stosowaniu funkcji zmiany promienia z całkowitymi wartościami. Tak wysokie błędy topograficzne dyskwalifikują możliwość zastosowania zaproponowanej funkcji trójkątnej do użycia na zbiorze Cardiotocography.

Zarówno zastosowanie funkcji progowej jak i eksponencjalnej przyniosło podobne rezultaty. Błędy topograficzne miały mniejszą wartość w odniesieniu do funkcji trójkątnej, przy lekko większej wartości błędów kwantyzacji. Mapy zwycięzców jednak cechowały się dużo lepszym grupowaniem, choć nie aż tak dobrym, jak dla funkcji Gaussa. W przypadku tych funkcji można stwierdzić, że wybór całkowitego promienia spowodował większe wartości błędów kwantyzacji i

topograficznych.

Zastosowanie funkcji Gaussa okazało się najtrafniejszym wyborem, w kontekście jednoczesnego zmniejszania błędu kwantyzacji i topograficznego. Mapa zwycięzców dla funkcji Gaussa z zastosowaniem funkcji z niecałkowitą zmianą promienia oraz topologią heksagonalną przedstawiona została na rysunku 5.2. Wybór funkcji Gaussa przełożył się na najbardziej precyzyjne grupowanie danych, w odniesieniu do innych zastosowanych funkcji.

W badaniu na zbiorze Cardiotocography zastosowanie topologii heksagonalnej prowadziło do znacznego zmniejszania błędu topograficznego, niekiedy kosztem minimalnego wzrostu błędu kwantyzacji. Zauważono również, że analiza wartości błędu topograficznego jest bardziej miarodajna w kontekście interpretacji grupowania za pomocą mapy zwycięzców, niż wartość błędu kwantyzacji. Niższy błąd topograficzny skutkuje zdecydowanie lepszym grupowaniem, nawet w przypadku większego błędu kwantyzacji.

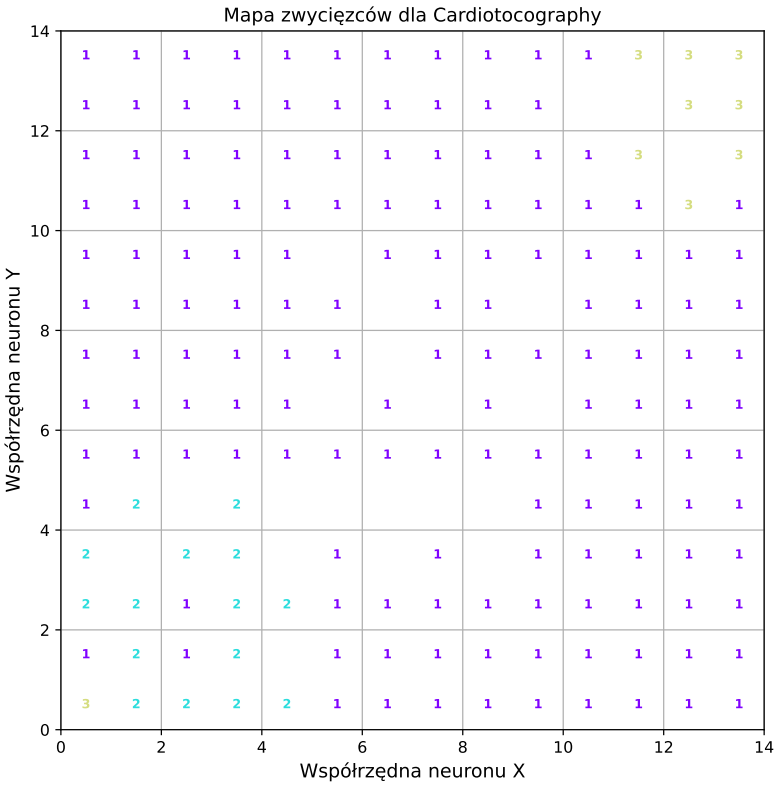
W przeprowadzonym badaniu wybór niecałkowitego promienia wpłynął przede wszystkim na wartość błędu topograficznego, znacząco zmniejszając jego wartość w porównaniu do konfiguracji z promieniem przyjmującym wyłącznie wartości całkowite.

Grupowanie pokazane na rysunku 5.2 daje zadowalające wyniki, choć nie bardzo dobre. Zdarzają się etykiety danych ”podejrzanych” pośród wyników ”normalnych”. Również jedna z danych opatrzona etykietą ”patologiczne” została zaklasyfikowana łącznie z danymi ”podejrzanymi”. Wyniki te sugerują, że sieć SOM z algorytmem WTM może być użyteczna w klasyfikacji tego typu danych, jednak dla pełniejszej skuteczności konieczne byłoby zastosowanie innej techniki klasyfikacyjnej, lepiej dostosowanej do specyfiki danych. Stosowanie wyłącznie sieci Kohonena może być narzędziem jedynie wspomagającym analizę i klasyfikacje danego przypadku. Przy podjęciu ostatecznej decyzji kluczowa powinna być jednak ocena specjalisty, szczególnie w tak wrażliwej dziedzinie, jak klasyfikacja danych medycznych.

5.4. Badania na zbiorze MNIST

Badaniami objęto również zbior MNIST (ang. ”*Modified National Institute of Standards and Technology dataset*”, tłumaczenie: ”zmodyfikowany zbiór danych Narodowego Instytutu Standardów i Technologii”) [22]. Został on stworzony w 1998 roku, jako zmodyfikowana wersja zbioru NIST. Jest to jeden z najbardziej popularnych zbiorów danych przeznaczonych do klasyfikacji i klasteryzacji. Zbiór ten zawiera 70 000 obrazów ręcznie pisanych cyfr w skali szarości, o rozmiarze 28x28 pikseli. Każdy obraz przypisany jest do jednej z dziesięciu klas, odpowiadających cyfrom od 0 do 9. W tym zbiorze etykiety reprezentują wartości numeryczne przypisane do każdego obrazu, jednoznacznie identyfikując przedstawioną na nim cyfrę (od 0 do 9). W wyniku wykorzystania skali szarości, po spłaszczeniu obrazu do jednowymiarowego wektora każda z danych ma 784 wymiary.

Decyzja o przeprowadzeniu badań na tym zbiorze wynikała z konieczności zbadania możliwości stosowania sieci SOM w zbiorach danych z dużą liczbą wzorców wejściowych, w których każdy wektor wejściowy ma wiele cech. Rozłożenie liczebności w klasach jest równomierne, jednak istnieje możliwość błędnej klasyfikacji danych, ze względu na ich stopień podobieństwa. Przy tak wysokiej liczbie cech istnieje bardzo duże prawdopodobieństwo błędnej klasyfikacji. Może to wynikać między innymi z różnic w stylach pisma i podobieństwa niektórych cyfr (np. ”3” i ”8” lub ”4” i ”9”). Badanie może również pokazać zdolności do zachowywania topologii wielowymiarowych danych wejściowych w tak dużym i zróżnicowanym zbiorze danych. W taki przypadku oczekuje się, że na mapie zwycięzców etykiety liczbowe danych podobnych do siebie wizualnie będą znaj-



Rysunek 5.2: Mapa zwycięzców otrzymana w wyniku testów na zbiorze Cardiotocography

dowały się w niedalekiej odległości (na przykład klasa z etykietą ”2” powinna być zlokalizowana w pobliżu klasy z etykietą ”8”).

Trening sieci przeprowadzono dla siatek 30 na 30 neuronów, z początkowym promieniem równym 15 i początkową wartością współczynnika uczenia równą 0.5. Wykorzystano funkcję dążącą do jedności dla zmiany wartości promienia 4.2 oraz funkcję wymierną dla zmiany wartości współczynnika uczenia 4.1. Każda sieć była trenowana przez 200 epok. Z uwagi na wysoką liczbę neuronów w sieci przy procesie treningu skorzystano z zaimplementowanej możliwości obliczeń używających tensorów, wykorzystując przy tym procesor graficzny.

Wyniki otrzymanych błędów kwantyzacji i topograficznych przedstawiono w tabeli 5.3.

5.4.1. Wnioski z badań na zbiorze MNIST

Dla zbioru MNIST, z uwagi na dużą liczbę wymiarów wektorów wejściowych, maksymalny błąd kwantyzacji, wyznaczony za pomocą zależności 3.17, przyjmuje wartość $\sqrt{784} = 28$. Średni błąd kwantyzacji dla wszystkich badań wyniósł 5.033, co stanowi 17.9% wartości maksymalnej. Jest to wynik relatywnie wysoki, jednak spodziewany, ze względu na dużą ilość danych uczących, z wieloma wymiarami. Miara ta jednak nie skreśla zastosowania sieci SOM przy klasyfikowaniu takich obrazów, jak te w zbiorze MNIST. Wartości błędów kwantyzacji nie różniły się znacząco miedzy sobą, niezależnie od zastosowanej funkcji sąsiedztwa, jednak błędy topograficzne miały zupełnie inne wartości, w zależności od zastosowanej funkcji sąsiedztwa.

Zastosowanie funkcja progowej doprowadziło do największych błędów topograficznych.

Tabela 5.3: Wartości błędów topograficznych i kwantyzacji z testów na zbiorze MNIST

funkcja sasiedztwa	Rodzaj błędu	hex$_{ncp}$	rect$_{ncp}$	hex$_{cp}$	rect$_{cp}$
gaussa	błąd kwantyzacji	4.890	4.953	4.906	5.022
	błąd topograficzny	0.039	0.113	0.064	0.127
trójkątna	błąd kwantyzacji	4.913	4.954	4.913	5.236
	błąd topograficzny	0.053	0.121	0.067	0.149
prostokątna	błąd kwantyzacji	5.110	5.221	4.923	5.334
	błąd topograficzny	0.114	0.242	0.151	0.352
eksponencjalna	błąd kwantyzacji	4.952	4.982	4.915	5.312
	błąd topograficzny	0.073	0.125	0.062	0.144

Lepsze rezultaty osiągnięto przy niecałkowitej zmianie promienia sąsiedztwa, jednak nie są to na tyle niskie wartości, aby umożliwić poprawną klasyfikację obrazów przedstawiających liczby. Również mapy zwycięzców dla tej funkcji cechowały się niepoprawnym grupowaniem i zupełnym niezachowywaniem relacji w danych wejściowych.

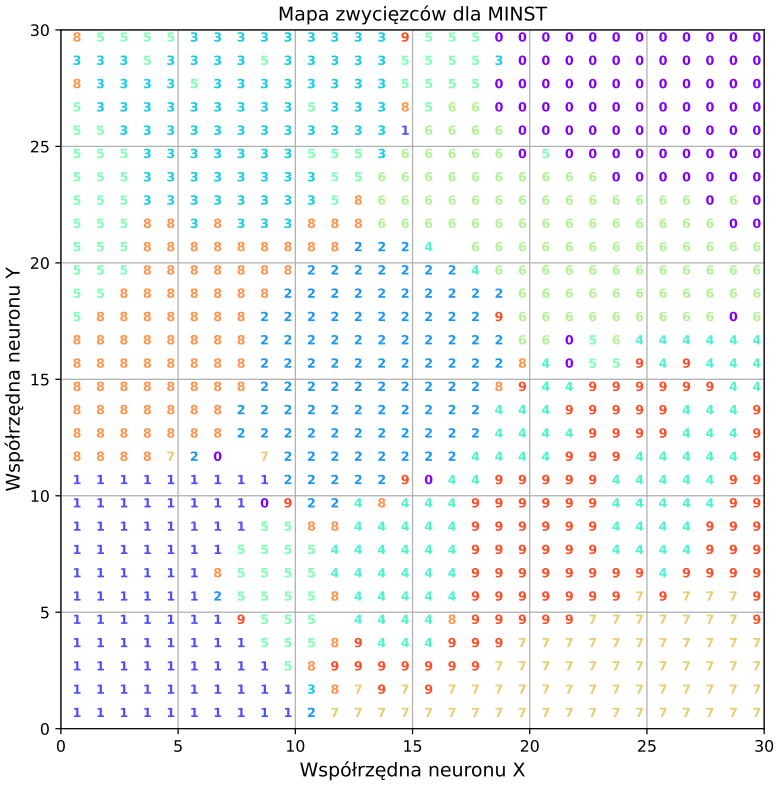
Badania z wykorzystaniem funkcji trójkątnej i funkcji eksponencjalnej skutkowały bardzo zbliżonymi wartościami błędów, zarówno kwantyzacji jak i topograficznych. Z tych dwóch podejść jednak uzyskano nieznacznie lepsze rezultaty na mapie zwycięzców, wykorzystując funkcję trójkątną. Podobnie jak przy zastosowaniu funkcji progowej, w tych wypadkach również lepiej sprawdzały się topologie heksagonalne, wraz z niecałkowitą zmianą promienia. Mapy zwycięzców dla przypadków stosowania funkcji trójkątnej były w ogólności bardziej precyzyjne niż te stworzone za pomocą sieci trenowanych funkcją eksponencjalną. Mapy te były bardzo zbliżone do najlepszych wyników, uzyskanych za pomocą funkcji Gaussa.

Najniższe wartości błędów kwantyzacji i topograficznych uzyskano dla zastosowania funkcji Gaussa. Najlepsze grupowanie uzyskano, stosując w funkcje zmiany promienia przyjmującą wartości niecałkowite oraz topologie heksagonalną. Mapę zwycięzców dla tego przypadku przedstawiono na 5.3. W innych przypadkach osiągano zbliżone wartości błędów kwantyzacji, jednak błędy topograficzne istotnie się różniły. Osiągnięto lepsze rezultaty stosując topologię heksagonalną.

Z badań przeprowadzonych na zbiorze MNIST można wyciągnąć pewne wnioski. Stosowanie niecałkowitych wartości promienia prowadzi do zmniejszenia wartości błędów topograficznych przy przypisywaniu danych z tego zbioru do klas. Jest to niezależne od zastosowanej funkcji sąsiedztwa. Dodatkowo, najlepsze rezultaty osiągnięto za pomocą funkcji Gaussa, co jest zgodne z trendami badań w literaturze [11, 18]. Niewiele gorsze rezultaty uzyskano za pomocą funkcji trójkątnej i eksponencjalnej, przy czym pierwsza z nich dała nieznacznie lepsze rezultaty. Funkcja prostokątna nie umożliwiła poprawnego przypisania do klas danych ze zbioru MNIST.
W przypadku rozważania wpływu topologii sieci na wartość błędów oraz na jakość mapy zwycięzców, można stwierdzić, że zastosowanie topologii heksagonalnej poprawiło efekty wszystkich miar. Może być to powiązane ze specyficzną strukturą danych ze zbioru MNIST.

Przedstawiona na rysunku 5.3 mapa zwycięzców poprawnie odwzorowuje strukturę danych wejściowych, wykazując widoczne lokalne skupiska odpowiadające określonym klasom, takim jak 3, 7, 2 czy 0. Niemniej jednak, w niektórych obszarach pojawia się mieszanie klas lub brak jednoznacznego przypisania. Topologia danych została zachowana w znacznym stopniu, choć

można również zauważyć błędy w przypisaniu. Rezultaty te oznaczają, że stosowanie jedynie sieci SOM, bez innych narzędzi analizy danych, nie gwarantuje idealnego przypisywania do klas tak złożone dane jak obrazy rozmiaru 28x28.



Rysunek 5.3: Mapa zwycięzców otrzymana w wyniku testów na zbiorze MNIST

6. PODSUMOWANIE

Samoorganizujące się sieci neuronowe mogą być wykorzystywane jako narzędzie do klasyfikacji danych rozmaitych typów, w szerokim spektrum zastosowań, w tym w takiej dziedzinie jak automatyka i robotyka. Mogą stanowić zarówno element większych systemów rozpoznawania wzorców, jak i samodzielne narzędzie do przypisywania klas w zbiorach danych z ograniczoną liczbą wymiarów.

W rozdziale tym przedstawiono wnioski z badań, opisanych w rozdziale 5, oraz wskazano potencjalny kierunek dalszych prac, mających na celu poprawienie osiągniętych rezultatów.

6.1. Wnioski z przeprowadzonych badań

Przeprowadzone badania miały na celu ocenę wpływu różnych funkcji sąsiedztwa na skuteczność sieci samoorganizujących się, w zadaniach wstępnej klasyfikacji (czyli klasteryzacji) danych. Testy przeprowadzono na trzech ogólnodostępnych zbiorach danych: IRIS, Cardiotocography i MNIST, różniących się licznością, liczbą wymiarów, stopniem złożoności oraz rozkładem danych w klasach, od równomierności, po znaczącą dominację jednej z klas. Kluczowymi metrykami oceny jakości klasteryzacji były wartości błędu kwantyzacji i błędu topograficznego. Jako narzędzie pomocnicze użyto również mapy zwycięzców, służącej do wizualizacji wyników. Na podstawie uzyskanych wyników można sformułować wstępne wnioski odnośnie możliwości klasyfikacji za pomocą sieci SOM, jednak należy pamiętać, że badania zostały przeprowadzone na ograniczonej liczbie zbiorów danych. Aby wyciągnąć bardziej ogólne wnioski, konieczne jest przeprowadzenie dalszych badań dla większej ilości zbiorów uczących.
Z przeprowadzonych badań można wyciągnąć następujące wnioski:

1. **Użycie funkcji Gaussa pozwoliło na minimalizację wartości błędu topograficznego**. Dla wszystkich zbiorów danych, wykorzystanych w badaniach, użycie funkcji Gaussa prowadziła do najniższych wartości błędów topograficznych. Zależność ta jest zgodna z dominującymi kierunkami badań nad sieciami Kohonena, ponieważ funkcja ta jest najczęściej używana [18]. Uzyskiwane w tych próbach błędy kwantyzacji były również niewielkie, a mapy zwycięzców cechowały się największą spójnością. Wskaźniki te mogą sugerować, że funkcja Gaussa jest najwłaściwszym wyborem przy próbie klasyfikacji wybranych danych za pomocą sieci SOM.

2. **Funkcja prostokątna nie umożliwia skutecznego przypisywania do klas w bardziej złożonych zbiorach danych**. Funkcja prostokątna uzyskiwała akceptowalne wartości błędów dla zbiorów IRIS i Cardiotocography, jednak przy bardziej złożonych danych, z większą liczbą wymiarów, takich jak te w zbiorze MNIST, osiągała najwyższe wartości błędów topograficznych i kwantyzacji. Doprowadziło to do błędnego przypisywania klas i braku uporządkowania na mapie zwycięzców. Oznacza to, że stosowanie funkcji progowej może być rozważane tylko przy danych z niską liczbą wymiarów i nieskomplikowaną topologią.

3. **Sieci z topologią heksagonalną uzyskiwały lepsze rezultaty od tych z prostokątną z ośmioma sąsiadami**. W każdym zbiorze danych stosowanie topologii heksagonalnej prowadziło do zmniejszenia wartości błędów topograficznych, a także często błędów kwantyzacji. Jako, że wynikać to może z wyboru zbiorów danych, to spostrzeżenie to warto uwia-

rygodnić poprzez przeprowadzenie dalszych badań.

4. **Zastosowanie promienia sąsiedztwa przyjmującego wartości niecałkowite zmniejszało błąd topograficzny**. W każdym z badanych przypadków zmiana promienia w sposób niecałkowity prowadziła do lepszego odwzorowania topologii danych, co przełożyło się na niższe błędy topograficzne. Niekiedy zwiększało to błąd kwantyzacji, jednak w niewielkim stopniu.

5. **Użyta postać funkcji trójkątnej nie sprawdziła się w zadaniach z prostymi zbiorami danych**. W badaniach funkcja trójkątna osiągnęła niskie wartości błędów kwantyzacji, jednak uzyskane błędy topograficzne były wysokie, szczególnie w przypadku zbiorów danych z niewielką liczbą wymiarów. Generowane mapy zwycięzców były niepoprawne, co wskazuje na problemy z zachowaniem topologii danych. W bardziej złożonych zbiorach, funkcja trójkątna osiągała lepsze rezultaty, nadal jednak ustępując funkcji Gaussa. Wyniki wskazują, że zastosowana funkcja trójkątna ma ograniczenia w zadaniach wymagających precyzyjnego odwzorowania struktury danych. W literaturze dostępne są bardziej zaawansowane warianty tej funkcji (między innymi ten zaproponowany w pracy [4]), uwzględniające na przykład zmienne nachylenie jej zbocza. Tego typu rozwiązania, przypominające kształtem charakterystyki funkcję Gaussa, mogą prowadzić do znacznie mniejszych wartości błędów. Należałoby jednak przeprowadzić dalsze badania mające na celu weryfikację możliwej poprawy wyników.

6. **Sieci SOM są skuteczne w zachowywaniu topologii danych, szczególnie przy niskiej liczbie wymiarów**. Użycie sieci Kohonena prowadziło do bardzo dobrego odwzorowania struktury danych, zwłaszcza w prostych zbiorach o niewielkiej liczbie wymiarów, takich jak IRIS. Wygenerowane mapy zwycięzców zachowały zależności topologiczne, ponieważ były na nich widoczne spójne klastry. Jednocześnie zaproponowane modele osiągnęły akceptowalne wyniki w bardziej złożonych zbiorach, jednak przy takich danych efekty klasyfikacji nie były bezbłędne.

7. **Sieci SOM wymagają dodatkowych narzędzi przy klasyfikacji złożonych danych**. W przypadku zbiorów o dużej liczbie wymiarów (np. MNIST) sieci SOM z funkcją Gaussa mogły z powodzeniem tworzyć w miarę poprawne mapy zwycięzców, ale dla pełnej skuteczności klasyfikacji danych konieczne byłoby zastosowanie również innych narzędzi. Podczas badań na zbiorze MNIST, dane podobnych cyfr, takich jak „4” i „9”, były w niektórych przypadkach błędnie przypisywane do tych samych obszarów, co sugeruje konieczność wsparcia SOM przez inne narzędzia klasyfikacyjne.

6.2. Potencjalne kierunki dalszych prac

W celu poprawy osiągniętych wyników rozważyć należy podjęcie dalszych działań. Istnieje możliwość zastosowania innej metody implementacji algorytmu WTM, tak zwanego algorytmu wsadowego (ang. *batch learning*) [25] lub dwufazowego [5]. Kolejnym sposobem poprawy może być użycie technik redukcji wymiarów przed przetworzeniem danych przez sieć Kohonena, na przykład zastosowanie takiego narzędzia jak analiza głównych składowych, PCA (ang. principal component analysis). W celu polepszenia wyników można również zbadać system składający się z sieci SOM uczonej za pomocą algorytmu WTM w połączeniu z innym modelem głębokiego uczenia [18]. Alternatywnie należy brać pod uwagę zastosowanie innego algorytmu do uczenia sieci samoorganizującej się, takiego jak np. gaz neuronowy.

SPIS RYSUNKÓW

2.1 Schemat budowy komórki nerwowej, z pracy [4] 11
2.2 Schemat modelu sztucznego neuronu 11
2.3 Schemat przepływu informacji w sieciach FNN, [1] 14
2.4 Schemat przepływu informacji w sieciach RNN, [3] 15

3.1 Architektura sieci SOM 19
3.2 Przykłady topologi sieci SOM: a) topologia prostokątna z czterema sąsiadami (Rect4), b) topologia prostokątna z ośmioma sąsiadami (Rect8), c) topologia hexagonalna. 22
3.3 Schemat blokowy algorytmu WTM 24

4.1 Opis przebiegu algorytmu uczenia SOM 34
4.2 Wykresy zmian promienia sąsiedztwa oraz współczynnika uczenia 37

5.1 Mapa zwycięzców otrzymana w wyniku testów na zbiorze IRIS 41
5.2 Mapa zwycięzców otrzymana w wyniku testów na zbiorze Cardiotocography . . . 44
5.3 Mapa zwycięzców otrzymana w wyniku testów na zbiorze MNIST 46

SPIS TABEL

5.1 Wartości błędów topograficznych i kwantyzacji z testów na zbiorze IRIS 40
5.2 Wartości błędów topograficznych i kwantyzacji z testów na zbiorze Cardiotocography 42
5.3 Wartości błędów topograficznych i kwantyzacji z testów na zbiorze MNIST 45

BIBLIOGRAFIA

[1] J. M. Żurada, M. Barski, and W. Jędruch. *Sztuczne sieci neuronowe: podstawy teorii i zastosowania*. Wydawnictwo Naukowe PWN, Warszawa, 1996.

[2] Ryszard Tadeusiewicz and Akademicka Oficyna Wydawnicza RM Wydawca. *Sieci neuronowe / Ryszard Tadeusiewicz.* Problemy Współczesnej Nauki i Techniki. Informatyka. Akademicka Oficyna Wydawnicza RM, Warszawa, [wyd. 2]. edition, 1993.

[3] Stanisław Osowski. *Sieci neuronowe w ujęciu algorytmicznym*. Wydawnictwa Naukowo-Techniczne, Warszawa, 2 edition, 1997.

[4] Marta Kolasa. *Uczenie się samoorganizujących map Kohonena metodą WTM implmentowaną sprzętowo*. PhD thesis, Uniwersytet Technologiczno-Przyrodniczy im. J.J. Śniadeckich w Bydgoszczy, Wydział Telekomunikacji i Elektrotechniki, 2011.

[5] Simon S. Haykin and Pearson. *Neural networks and learning machines / Simon Haykin.* Pearson Education, Upper Saddle River, 3rd ed., interantional edition. edition, 2009.

[6] Vincenzo Laveglia and Edmondo Trentin. Downward-growing neural networks. *Entropy (Basel, Switzerland)*, 25(5):733–, 2023.

[7] Marc M. Van Hulle. Self-organizing maps. In *Handbook of Natural Computing*, pages 585–622. Springer Berlin / Heidelberg, Germany, 2012.

[8] Teuvo Kohonen. *Self-Organizing Maps*. Springer-Verlag, 1995.

[9] Tomasz Ciechulski and Stanisław Osowski. Zastosowanie sieci neuronowych kohonena do prognozowania obciążeń elektroenergetycznych. *Przegląd Elektrotechniczny*, 92(10):209–213, 2016.

[10] Sokchoo Ng and Mieowkee Chan. Effect of neighbourhood size selection in som-based image feature extraction. *International Journal of Machine Learning and Computing*, 9(2):195–200, 2019.

[11] Marta Kolasa. An energy-efficient, parallel neighborhood and adaptation functions for hardware implemented self-organizing maps applied in smart grid. *Energies (Basel)*, 13(5):1197–, 2020.

[12] J. Pena, M. Vanegas, and A. Valencia. Digital hardware architectures of kohonen's self organizing feature maps with exponential neighboring function. In *2006 IEEE International Conference on Reconfigurable Computing and FPGA's (ReConFig 2006)*, pages 1–8. IEEE, 2006.

[13] Julius Venskus, Povilas Treigys, Jolita Bernatavičienė, Viktor Medvedev, Miroslav Voznak, Mindaugas Kurmis, and Violeta Bulbenkienė. Integration of a self-organizing map and a virtual pheromone for real-time abnormal movement detection in marine traffic. *Informatica (Vilnius, Lithuania)*, 28(2):359–374, 2017.

[14] Rafal Dlugosz, Tomasz Talaska, Witold Pedrycz, and Ryszard Wojtyna. Realization of the conscience mechanism in cmos implementation of winner-takes-all self-organizing neural networks. *IEEE transactions on neural networks*, 21(6):961–971, 2010.

[15] T.M. Martinetz, S.G. Berkovich, and K.J. Schulten. 'neural-gas' network for vector quantization and its application to time-series prediction. *IEEE transactions on neural networks*, 4(4):558–569, 1993.

[16] Karolina Bartos. Sieć som jako przykład sieci samoorganizującej się. In *Sieć SOM jako przykład sieci samoorganizującej się*. Wydawnictwo Uniwersytetu Ekonomicznego we Wrocławiu, 2012.

[17] Surjeet Dalal, Umesh Kumar Lilhore, Poongodi Manoharan, Uma Rani, Fadl Dahan, Fahima Hajjej, Ismail Keshta, Ashish Sharma, Sarita Simaiya, and Kaamran Raahemifar. An efficient brain tumor segmentation method based on adaptive moving self-organizing map and fuzzy k-mean clustering. *Sensors*, 23(18), 2023.

[18] Chathurika S Wickramasinghe, Kasun Amarasinghe, and Milos Manic. Deep self-organizing maps for unsupervised image classification. *IEEE Transactions on Industrial Informatics*, 15(11):5837–5845, 2019.

[19] Giuseppe Vettigli. Minisom: minimalistic and numpy-based implementation of the self organizing map, 2018.

[20] Mark Hudson Beale, Martin T Hagan, and Howard B Demuth. Deep learning toolbox. *R2018b User's Guide; The MathWorks, Inc.: Natick, MA, USA*, 2018.

[21] Ashish Ranjan Jha. *Mastering PyTorch: Create and deploy deep learning models from CNNs to multimodal models, LLMs, and beyond*. Packt Publishing, Birmingham, 2 edition, 2024.

[22] Li Deng. The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, 29(6):141–142, 2012.

[23] R. A. Fisher. Iris. UCI Machine Learning Repository, 1936. DOI: https://doi.org/10.24432/C56C76.

[24] D. Campos and J. Bernardes. Cardiotocography. UCI Machine Learning Repository, 2000. DOI: https://doi.org/10.24432/C51S4N.

[25] Iván Machón-González, Hilario López-García, and Josá Luís Calvo-Rolle. A hybrid batch som-ng algorithm. In *The 2010 international joint conference on neural networks (IJCNN)*, pages 1–5. IEEE, 2010.

A. INSTRUKCJA DLA UŻYTKOWNIKA

Programy w języku Python, zawierające implementację klas do obsługi logiki oraz wizualizacji wyników znajdują się w archiwum ZIP, dołączonym do niniejszej pracy. W celu poprawnego działania należy pobrać ten folder i go rozpakować. Aby powtórzyć badania opisane w pracy należy następnie podjąć następujące kroki:

1. Pobrać wybrany zbiór uczący
2. Znormalizować dane ze zbioru
3. Podzielić zbiór na dane uczące i testowe
4. Stworzyć obiekt klasy SelfOrganizingMap z poprawnym doborem parametrów
5. Przeprowadzić trening, wykorzystując metodę train
6. Przeprowadzić ewaluacje modelu

Do pobrania zbiorów uczących można użyć pomocniczych pakietów, takich jak `scipy`, choć nie jest to konieczne. Można też skorzystać z źródeł internetowych, z których można bezpośrednio pobrać pliki z danymi. W badaniach normalizację przeprowadzono za pomocą zależności podanej w rozdziale 5, jednak można również skorzystać metody MinMaxScaller z `scipy`. Przykładowe tworzenie obiektu klasy przedstawiono na A.1. Pod zmienne należy podstawić wybrane parametry sieci SOM.

```
som = SelfOrganizingMap(
    dim_x=DIM_X, dim_y=DIM_Y, num_of_iterations=NUM_OF_EPOCHS, input_dimension=DIM,
    distance_metric=WYBRANA_METRYKA, initial_radius=INITIAL_R,
    initial_learning_rate=LEARNING_RATE,
    network_topology= WYBRANA_TOPOLOGIA,
    neighbourhood_function=SASIEDZTWO,
    int_radius=RADIUS_FLAG
)
```

Listing A.1: Przykład inicjalizacji klasy `SelfOrganizingMap`

W celu uruchomienia programu niezbędne jest posiadanie pakietów, wraz z odpowiednią wersją Pythona. Wszystkie wymagane biblioteki, wraz z użytą wersją Pythona, dostępne są w pliku `requirements.txt`. Zaleca się tworzenie wirtualnego środowiska za pomocą narzędzia `venv` oraz instalowanie pakietów z pomocą menadżera `pip`. W celu instalacji poprawnej wersji pakietu Pytorch należy skorzystać ze strony internetowej pakietu Pytorch. Spowodowane jest to koniecznym wyborem wersji pakietu, zależnej od dostępnych środowisk pracy, takich jak wersja Pythona czy obsługiwane sterowniki GPU. Ewaluacji modelu można dokonywać za pomocą analizy błędu kwantyzacji i błędu topograficznego, korzystając z metod `quantization_error` oraz `topographic_error`. Dodatkowo, możliwe jest wyświetlenie mapy zwycięzców za pomocą metody `make_winmap` z klasy `VisualizeSOMResults`.